Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Institut International de Technologie à Sfax
وزارة التعليم العالي والبحث العلمي
المدرسة العليا الدولية الخاصة للتكنولوجيا بصفاقس

PROJET DE TEST ET QUALITÉ LOGICIEL

Présenté par

Chaima MAATAR

Event Planner

Année Universitaire : 2025-2026

SOMMAIRE

Introduction..... **2**

I. Mise en place de l'analyse statique..... **2**

II. Cas de test documentés..... **3**

a. Inscription d'un utilisateur au site..... 3

b. Connexion d'un utilisateur au site..... 15

c. Gestion des catégories et des événements par l'administrateur..... 19

d. Blocage des actions administrateur sans authentification..... 26

e. Unicité du nom de catégorie..... 31

f. Compréhensibilité des messages d'erreur..... 35

g. Protection CSRF des formulaires..... 38

III. Rapport final..... **42**

a. Fonctionnalités administrateur..... 42

b. Fonctionnalités utilisateur..... 43

c. Tests non fonctionnels..... 43

INTRODUCTION

Dans le cadre du module Test et Qualité Logiciel, ce projet a pour objectif de mettre en pratique les méthodologies et techniques de test sur une application web: Event Planner. Cette application développée avec le framework Laravel permet la gestion complète d'événements avec deux types d'utilisateurs (administrateurs et utilisateurs standards).

I. MISE EN PLACE DE L'ANALYSE STATIQUE

Pour assurer la qualité du code source et détecter les erreurs potentielles avant l'exécution, nous avons implémenté des tests statiques utilisant PHPStan, un analyseur statique puissant pour PHP, combiné à Larastan pour une meilleure intégration avec le framework Laravel.

```
PS D:\xampp82\htdocs\projet> composer require --dev phpstan/phpstan
./composer.json has been updated
Running composer update phpstan/phpstan
Loading composer repositories with package information
Updating dependencies
PS D:\xampp82\htdocs\projet> composer require --dev nunomaduro/larastan
./composer.json has been updated
Running composer update nunomaduro/larastan
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
```

Figure 1 : Installation et configuration de PHPStan et Larastan

La configuration a été complétée par la création d'un fichier phpstan.neon contenant les paramètres d'analyse :

```
PS D:\xampp82\htdocs\projet> New-Item phpstan.neon -ItemType File
>>
>>


    Directory: D:\xampp82\htdocs\projet


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         15-Jan-26  10:17 PM              0 phpstan.neon
```

Figure 2 : Création du fichier de configuration phpstan.neon

phpstan.neon

```
1   includes:
2       - ./vendor/nunomaduro/larastan/extension.neon
3
4   parameters:
5       paths:
6           - app
7       level: 5
8
```

Figure 3 : configuration du fichier de configuration phpstan.neon

L'exécution de l'analyse statique a produit des résultats satisfaisants :

```
                          .\vendor\bin\phpstan analyse
>> D:\xampp82\htdocs\projet>
Note: Using configuration file D:\xampp82\htdocs\projet\phpstan.neon.
 13/13 [============================] 100%


 [OK] No errors

PS D:\xampp82\htdocs\projet>
```

Figure 4 : Résultat de l'analyse statique - Aucune erreur détectée

II. CAS DE TEST DOCUMENTES

a. Documentation de Cas de Test : Inscription d'un utilisateur au site

❖ Description Générale

En tant qu'utilisateur non connecté, Je veux créer un compte, Afin d'accéder aux fonctionnalités réservées du site de gestion des événements (inscription aux événements, réservation, gestion du profil).

Ce cas de test vise à vérifier la validation et la non-validation des données lors de l'inscription d'un nouvel utilisateur via le formulaire /register.

Titre	Inscription d’un utilisateur au site
Description	Vérifier qu’un utilisateur non connecté peut créer un compte avec des données valides et que le système refuse les données invalides
Technique de test	**Boîte noire** **Motivation du choix :** Le test est réalisé du point de vue utilisateur, sans connaissance de l’implémentation interne. L’objectif est de vérifier que le système réagit correctement aux différentes entrées utilisateur, conformément aux exigences fonctionnelles.
Niveau de test	**Test système** Le scénario couvre l’ensemble du flux : interface web → contrôleur → validation → base de données.
Type de test	Test fonctionnel
Prérequis	➢ Serveur Laravel démarré ➢ Base de données accessible ➢ Page d'inscription disponible à /register
Étapes	1. Accéder à /register 2. Remplir tous les champs avec données valides 3. Cliquer sur "Sign Up"
Automatisation	Le cas de test est automatisé avec Selenium (Python) afin de simuler le comportement réel d’un utilisateur via un navigateur (Chrome).
Règles de validation	➢ Nom entre 1 et 255 caractères ➢ Email valide, unique, entre 6 et 255 caractères ➢ Mot de passe entre 6 et 255 caractères, avec au moins une majuscule et un chiffre, confirmé

Tableau 1 : Fiche du cas de test « Inscription d’un utilisateur »

❖ Scénarios testés

Scénario	Résultat attendu	Résultat obtenu
CV1 - Classe valide : Tous champs valides	Inscription réussie, redirection vers page d'accueil	**PASS**
CI1 - Classe invalide : Email déjà existant	Message d'erreur, rester sur page d'inscription	**PASS**
CI2 - Classe invalide : Format email incorrect		**PASS**
CI3 - Classe invalide : Mot de passe sans majuscule		**FAIL → PASS**
CI4 - Classe invalide : Mot de passe sans chiffre		**FAIL → PASS**
CI5 - Classe invalide : Confirmation différente		**PASS**
CI6 - Classe invalide : Champs requis vides		**PASS**

Tableau 2 : Scénarios testés – inscription

Remarque : pour CI2, le champ email est de type « email » : c'est d'abord le navigateur (validation HTML5) qui bloque l'envoi du formulaire. La règle « email » de Laravel reste une seconde vérification côté serveur.

```
PS D:\xampp82\htdocs\projet> & D:/xampp82/htdocs/projet/.venv/Scripts/python.exe d:/xampp82/htdocs/projet/test/inscription.py
CV1: Tous les champs valides
[PASS] CV1 – Inscription réussie avec données valides
CI1: Email déjà existant
[PASS] CI1 – Email déjà existant → refusé correctement
CI2: Email invalide
[PASS] CI2 – Email invalide → refusé correctement
CI3: Mot de passe sans majuscule
[FAIL] CI3 – Mot de passe sans majuscule → accepté (ERREUR)
CI4: Mot de passe sans chiffre
[FAIL] CI4 – Mot de passe sans chiffre → accepté (ERREUR)
CI5: Confirmation mot de passe différente
[PASS] CI5 – Confirmation différente → refusée correctement
CI6: Champs requis manquants
[PASS] CI6 – Champs requis vides → refusé correctement
PS D:\xampp82\htdocs\projet>
```

Figure 5 : Résultats des tests de validation des données avant correction

```php
public function register(Request $request)
{
    $validated = $request->validate([
        'name' => ['required', 'string','min:1' ,'max:255'],
        'email' => ['required', 'email', 'unique:users,email','min:6'],
        'password' => ['required', 'min:6', 'confirmed','regex:/[A-Z]/','regex:/[0-9]/'],
    ]);
```

Figure 6 : Correction des règles de validation

```
CV1: Tous les champs valides
[PASS] CV1 – Inscription réussie avec données valides
CI1: Email déjà existant
[PASS] CI1 – Email déjà existant → refusé correctement
CI2: Email invalide
[PASS] CI2 – Email invalide → refusé correctement
CI3: Mot de passe sans majuscule
[PASS] CI3 – Mot de passe sans majuscule → refusé correctement
CI4: Mot de passe sans chiffre
[PASS] CI4 – Mot de passe sans chiffre → refusé correctement
CI5: Confirmation mot de passe différente
[PASS] CI5 – Confirmation différente → refusée correctement
CI6: Champs requis manquants
[PASS] CI6 – Champs requis vides → refusé correctement
```

Figure 7 : Résultats des tests de validation des données après correction

Métrique	Valeur
Total des cas de test définis	7
Total tests exécutés	7
Tests réussis initialement	5 (71.4%)
Tests échoués initialement	2 (28.6%)
Tests réussis après correction	7 (100%)
Taux d'exécution (exécutés / définis)	7/7(100%)

Tableau 3 : Statistiques des tests d'inscription (classes d'équivalence)

❖ Test des valeurs limites

Vérifier que le formulaire d'inscription applique correctement les contraintes de validation sur les champs name, email et password, en particulier aux bornes minimales et maximales autorisées, afin d'éviter les erreurs fonctionnelles et techniques.

Pour chaque test, seul le champ testé varie ; les autres champs reçoivent des valeurs valides. Les mots de passe générés contiennent une majuscule et un chiffre (par exemple « P1aaaa » pour 6 caractères), afin que seule la longueur soit testée et non les autres règles du mot de passe.

Champ	Intervalle valide	Valeurs testées :
name	[1, 255]	0, 1, 2, 254, 255, 256
email	[6, 255]	5, 6, 7, 254, 255, 256
password	[6, 255]	5, 6, 7, 254, 255, 256

Tableau 4 : Intervalles valides et valeurs testées

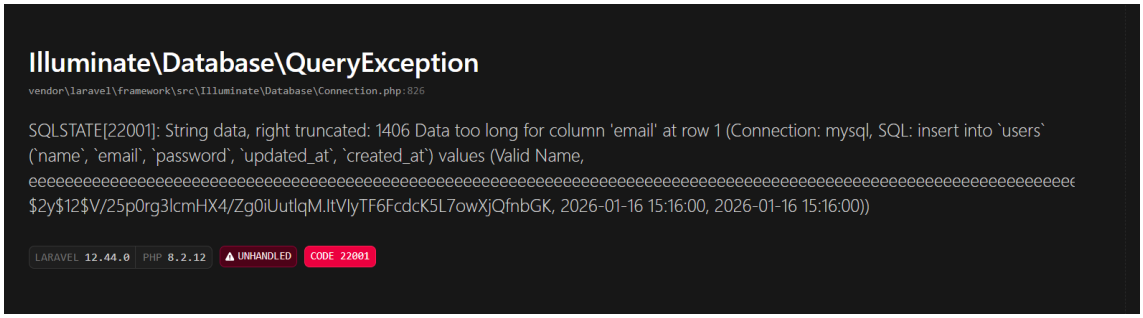
Champ	Valeur testée	Résultat attendu	Résultat initial	Résultat final	Pass/Fail
Nom	0 caractères	Reste à /register	Reste à /register	-	PASS
	1 caractère	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	2 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	254 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	255 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	256 caractères	Reste à /register	Reste à /register	-	PASS

Email	5 caractères	Reste à /register	Reste à /register	-	PASS
	6 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	7 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	254 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	255 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	256 caractères	Reste à /register	Erreur SQL (Data too long)	Reste à /register	FAIL → PASS
Password	5 caractères	Reste à /register	Reste à /register	-	PASS
	6 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	7 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	254 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	255 caractères	Passe à / (accueil)	Passe à / (accueil)	-	PASS
	256 caractères	Reste à /register	Passe à / (accueil)	Reste à /register	FAIL → PASS

Tableau 5 : Résultats des tests de valeurs limites

```
PS D:\xampp82\htdocs\projet> & D:/xampp82/htdocs/projet/.venv/Scripts/python.exe d:/xampp82/htdocs/projet/test/inscription.py
=== VALEURS LIMITES : NAME ===
Test name = 0
PASS
----------------------------------------
Test name = 1
PASS
----------------------------------------
Test name = 2
PASS
----------------------------------------
Test name = 254
PASS
----------------------------------------
Test name = 255
PASS
----------------------------------------
Test name = 256
PASS
----------------------------------------
=== VALEURS LIMITES : EMAIL ===
Test email = 5
PASS
----------------------------------------
Test email = 6
PASS
----------------------------------------
Test email = 7
PASS
----------------------------------------
Test email = 254
PASS
----------------------------------------
Test email = 255
PASS
----------------------------------------
Test email = 256
FAIL
----------------------------------------
=== VALEURS LIMITES : PASSWORD ===
Test password = 5
PASS
----------------------------------------
Test password = 6
PASS
----------------------------------------
Test password = 7
PASS
----------------------------------------
Test password = 254
PASS
----------------------------------------
Test password = 255
PASS
----------------------------------------
Test password = 256
FAIL
----------------------------------------
```

Figure 8 : Résultats initiaux des tests de valeurs limites


Illuminate\Database\QueryException
vendor\laravel\framework\src\Illuminate\Database\Connection.php:826
SQLSTATE[22001]: String data, right truncated: 1406 Data too long for column 'email' at row 1 (Connection: mysql, SQL: insert into `users` (`name`, `email`, `password`, `updated_at`, `created_at`) values (Valid Name,
eeee

```php
public function register(Request $request)
{
    $validated = $request->validate([
        'name' => ['required', 'string','min:1' ,'max:255'],
        'email' => ['required', 'email', 'unique:users,email','min:6','max:255'],
        'password' => ['required', 'min:6', 'confirmed','max:255'],
    ]);
```

Figure 10 : Correction de la validation des données côté backend

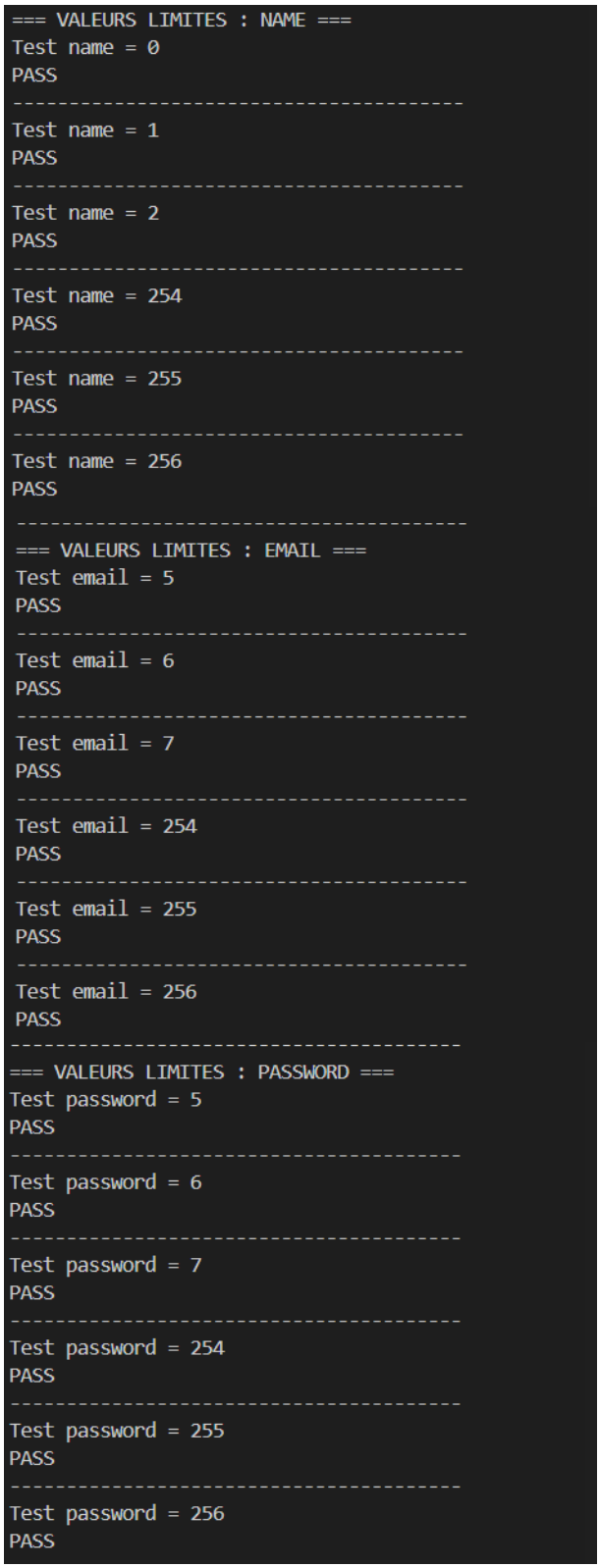


Figure 11 : Résultats des tests après correction

Métrique	Valeur
Total des cas de test définis	18
Total tests exécutés	18
Tests réussis initialement	16 (88.9%)
Tests échoués initialement	2 (11.1%)
Tests réussis après correction	18 (100%)
Taux d'exécution (exécutés / définis)	18/18 (100%)

Tableau 6 : Statistiques des tests de valeurs limites

❖ Script Selenium pour le test de l'inscription avec données valides, invalides et valeurs limites:

```
"""
Tests Selenium du formulaire d'inscription (/register) - Event Planner
 - classes d'équivalence (CV1, CI1..CI6)
 - valeurs limites (name, email, password)

Prérequis :
 - serveur Laravel démarré : php artisan serve
 - base de test : les comptes créés par une exécution précédente doivent être
   supprimés avant de relancer (les adresses des valeurs limites de l'email,
   ex. "e@t.fr", sont fixes et seraient refusées comme "déjà existantes").
"""
import time

from selenium import webdriver
from selenium.common.exceptions import TimeoutException
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

driver = webdriver.Chrome()
driver.maximize_window()

REGISTER_URL = "http://127.0.0.1:8000/register"
SUCCESS_URL = "http://127.0.0.1:8000/"

# Identifiant unique de l'exécution : évite le conflit "email déjà existant"
# quand on relance le script (sauf pour les emails très courts des valeurs
limites).
RUN = int(time.time())
```

```python
# ----------------------------
def soumettre(bouton):
    """Clique sur le bouton puis attend que la nouvelle page soit chargée.
    Si le navigateur bloque l'envoi (validation HTML5, ex. type="email"),
    la page ne change pas : on continue après le délai d'attente."""
    ancienne_page = driver.find_element(By.TAG_NAME, "html")
    bouton.click()
    try:
        WebDriverWait(driver, 5).until(EC.staleness_of(ancienne_page))
        WebDriverWait(driver, 5).until(
            lambda d: d.execute_script("return document.readyState") == "complete"
        )
    except TimeoutException:
        pass


def inscription(name, email, password, password_confirmation):
    driver.get(REGISTER_URL)

    driver.find_element(By.NAME, "name").clear()
    driver.find_element(By.NAME, "name").send_keys(name)

    driver.find_element(By.NAME, "email").clear()
    driver.find_element(By.NAME, "email").send_keys(email)

    driver.find_element(By.NAME, "password").clear()
    driver.find_element(By.NAME, "password").send_keys(password)

    driver.find_element(By.NAME, "password_confirmation").clear()
    driver.find_element(By.NAME,
"password_confirmation").send_keys(password_confirmation)

    soumettre(driver.find_element(By.XPATH, "//button[text()='Sign Up']"))


def deconnecter():
    # Après une inscription réussie l'utilisateur est connecté :
    # on supprime les cookies pour revenir à un visiteur non connecté.
    driver.delete_all_cookies()


# ----------------------------
# VALEURS LIMITES
def valeur_attendue(field, val):
    if field == "name":
        return 1 <= val <= 255
    if field == "email":
        return 6 <= val <= 255
    if field == "password":
```

```
        return 6 <= val <= 255


def test_valeur_limite(field, values):
    for val in values:
        # Valeurs valides par défaut : seul le champ testé varie
        name = "Valid Name"
        email = f"user_{RUN}_{field}_{val}@test.com"
        password = "Password123"

        if field == "name":
            name = "A" * val
        elif field == "email":
            email = ("e" * (val - 5)) + "@t.fr"          # longueur totale = val
        elif field == "password":
            # longueur = val, avec une majuscule et un chiffre :
            # seule la longueur est testée, pas les autres règles du mot de passe
            password = ("P1" + "a" * (val - 2)) if val >= 2 else "P" * val

        print(f"Test {field} = {val}")

        inscription(name, email, password, password)

        observe = driver.current_url == SUCCESS_URL
        attendu = valeur_attendue(field, val)
        erreur_technique = "QueryException" in driver.page_source or "Data too
long" in driver.page_source

        if observe == attendu and not erreur_technique:
            print("PASS")
        else:
            print("FAIL")
        print("-" * 40)

        deconnecter()


name_limits = [0, 1, 2, 254, 255, 256]
email_limits = [5, 6, 7, 254, 255, 256]
password_limits = [5, 6, 7, 254, 255, 256]

print("=== VALEURS LIMITES : NAME ===")
test_valeur_limite("name", name_limits)

print("=== VALEURS LIMITES : EMAIL ===")
test_valeur_limite("email", email_limits)

print("=== VALEURS LIMITES : PASSWORD ===")
test_valeur_limite("password", password_limits)
```

```python
# ----------------------------
# TESTS – CLASSES VALIDES
EMAIL_CV1 = f"chaima_{RUN}@test.com"

print("CV1: Tous les champs valides")
inscription("Chaima Maatar", EMAIL_CV1, "Password123", "Password123")
if driver.current_url == SUCCESS_URL:
    print("[PASS] CV1 – Inscription réussie avec données valides")
else:
    print("[FAIL] CV1 – Inscription échouée (ERREUR)")
deconnecter()

# ----------------------------
# TESTS – CLASSES INVALIDES
print("CI1: Email déjà existant")
inscription("Chaima Maatar", EMAIL_CV1, "Password123", "Password123")
if driver.current_url == REGISTER_URL:
    print("[PASS] CI1 – Email déjà existant → refusé correctement")
else:
    print("[FAIL] CI1 – Email déjà existant → accepté (ERREUR)")
deconnecter()

print("CI2: Email invalide")
inscription("Test User", "email_invalide", "Password123", "Password123")
if driver.current_url == REGISTER_URL:
    print("[PASS] CI2 – Email invalide → refusé correctement")
else:
    print("[FAIL] CI2 – Email invalide → accepté (ERREUR)")
deconnecter()

print("CI3: Mot de passe sans majuscule")
inscription("Test User", f"noupper_{RUN}@test.com", "password123", "password123")
if driver.current_url == REGISTER_URL:
    print("[PASS] CI3 – Mot de passe sans majuscule → refusé correctement")
else:
    print("[FAIL] CI3 – Mot de passe sans majuscule → accepté (ERREUR)")
deconnecter()

print("CI4: Mot de passe sans chiffre")
inscription("Test User", f"nodigit_{RUN}@test.com", "Password", "Password")
if driver.current_url == REGISTER_URL:
    print("[PASS] CI4 – Mot de passe sans chiffre → refusé correctement")
else:
    print("[FAIL] CI4 – Mot de passe sans chiffre → accepté (ERREUR)")
deconnecter()

print("CI5: Confirmation mot de passe différente")
```

```
inscription("Test User", f"diffpass_{RUN}@test.com", "Password123", "Password321")
if driver.current_url == REGISTER_URL:
    print("[PASS] CI5 – Confirmation différente → refusée correctement")
else:
    print("[FAIL] CI5 – Confirmation différente → acceptée (ERREUR)")
deconnecter()

print("CI6: Champs requis manquants")
inscription("", "", "", "")
if driver.current_url == REGISTER_URL:
    print("[PASS] CI6 – Champs requis vides → refusé correctement")
else:
    print("[FAIL] CI6 – Champs requis vides → accepté (ERREUR)")
deconnecter()

# ----------------------------
# FERMETURE DU DRIVER
driver.quit()
```

b. Documentation de Cas de Test : Connexion d'un utilisateur au site

❖ Description Générale

En tant qu'utilisateur enregistré, Je veux me connecter à mon compte, Afin d'accéder à mon espace personnel (consultation des événements, inscriptions, gestion du profil).

Ce cas de test vise à vérifier la validation et la non-validation des données lors de la connexion d'un utilisateur existant via le formulaire /login.

Titre	Connexion d'un utilisateur au site.
Description	Vérifier qu'un utilisateur enregistré peut se connecter avec des identifiants valides et que le système refuse l'accès en cas de données invalides ou manquantes.
Technique de test	**Boîte noire** **Motivation du choix :** Le test est réalisé du point de vue utilisateur, sans connaissance de l'implémentation interne. L'objectif est de vérifier que le système réagit correctement aux différentes combinaisons d'entrées utilisateur, conformément aux exigences fonctionnelles de sécurité et d'authentification.
Niveau de test	**Test système** Le scénario couvre l'ensemble du flux : interface web → contrôleur d'authentification → validation → session utilisateur.
Type de test	Test fonctionnel
Prérequis	➢ Serveur Laravel démarré ➢ Base de données accessible

	➢ Compte utilisateur déjà enregistré ➢ Page de connexion disponible à /login
Étapes	1. Accéder à la page /login 2. Saisir l'adresse email et le mot de passe 3. Cliquer sur le bouton "Sign In"
Automatisation	Le cas de test est automatisé avec Selenium (Python) afin de simuler le comportement réel d'un utilisateur via un navigateur (Chrome). Les différents scénarios sont exécutés automatiquement et les résultats sont affichés dans la console.
règles de validation	➢ Email obligatoire et au format valide ➢ Mot de passe obligatoire ➢ Les identifiants doivent correspondre à un compte existant

Tableau 7 : Fiche du cas de test « Connexion d'un utilisateur »

❖ Scénarios testés

Scénario	Résultat attendu	Résultat obtenu
CV1 – Classe valide : Email et mot de passe corrects	Connexion réussie, redirection vers l'espace personnel	**PASS**
CI1 – Classe invalide : Email valide + mot de passe incorrect	Refus de connexion, rester sur /login	**PASS**
CI2 – Classe invalide : Email au format incorrect		**PASS**
CI3 – Classe invalide : Champs requis vides		**PASS**

Tableau 8 : Scénarios testés – connexion

Remarque : pour CI2 (email « abc »), c'est le navigateur (validation HTML5 du champ de type « email ») qui bloque l'envoi du formulaire avant la requête au serveur.

```
(.venv) PS D:\xampp82\htdocs\projet>    python test\login_test.py
Test 1 : Classe valide
PASS – Connexion réussie
Test 2 : Email valide + mauvais mot de passe
PASS – Accès refusé
Test 3 : Email invalide
PASS – Email invalide rejeté
Test 4 : Champs vides
PASS – Champs vides rejetés
```

Figure 12 : Résultats des tests automatisés de connexion

Métrique	Valeur
Total des cas de test définis	4
Total tests exécutés	4
Tests réussis initialement	4 (100%)
Tests échoués initialement	0 (0%)
Tests réussis après correction	— (aucune correction nécessaire)
Taux d'exécution (exécutés / définis)	4/4 (100%)

Tableau 9 : Statistiques des tests de connexion

❖ Script Selenium pour le test de la connexion avec données valides et invalides :

```
"""
Tests Selenium du formulaire de connexion (/login) - Event Planner

Prérequis :
 - serveur Laravel démarré : php artisan serve
 - un compte utilisateur (non administrateur) user@test.com / Password123
   existe déjà (créé via /register)
"""
from selenium import webdriver
from selenium.common.exceptions import TimeoutException
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

driver = webdriver.Chrome()

BASE_URL = "http://localhost:8000/login"
HOME_URL = "http://localhost:8000/"


def soumettre(bouton):
    """Clique sur le bouton puis attend que la nouvelle page soit chargée.
    Si le navigateur bloque l'envoi (validation HTML5), la page ne change pas."""
    ancienne_page = driver.find_element(By.TAG_NAME, "html")
    bouton.click()
    try:
        WebDriverWait(driver, 5).until(EC.staleness_of(ancienne_page))
        WebDriverWait(driver, 5).until(
            lambda d: d.execute_script("return document.readyState") == "complete"
        )
    except TimeoutException:
```

```
        pass


def login(email, password):
    driver.get(BASE_URL)

    email_input = driver.find_element(By.NAME, "email")
    password_input = driver.find_element(By.NAME, "password")

    email_input.clear()
    password_input.clear()

    email_input.send_keys(email)
    password_input.send_keys(password)

    soumettre(driver.find_element(By.XPATH, "//button[text()='Sign In']"))


def deconnecter():
    # Après une connexion réussie, /login redirigerait vers l'accueil :
    # on supprime les cookies pour revenir à un visiteur non connecté.
    driver.delete_all_cookies()


# ------------------------------
# Classe valide
print("Test 1 : Classe valide")
login("user@test.com", "Password123")
if driver.current_url == HOME_URL:
    print("PASS — Connexion réussie")
else:
    print("FAIL — Connexion échouée")
deconnecter()

# ------------------------------
# Classe invalide 1
print("Test 2 : Email valide + mauvais mot de passe")
login("user@test.com", "wrongpass")
if "login" in driver.current_url:
    print("PASS — Accès refusé")
else:
    print("FAIL — Erreur de sécurité")
deconnecter()

# ------------------------------
# Classe invalide 2
print("Test 3 : Email invalide")
login("abc", "Password123")
if "login" in driver.current_url:
```

```
    print("PASS – Email invalide rejeté")
else:
    print("FAIL")
deconnecter()

# ----------------------------
# Classe invalide 3
print("Test 4 : Champs vides")
login("", "")
if "login" in driver.current_url:
    print("PASS – Champs vides rejetés")
else:
    print("FAIL")

driver.quit()
```

c. Documentation de Cas de Test : Gestion des catégories et des événements par l'administrateur :

❖ Description Générale

En tant qu'administrateur, Je veux gérer les catégories et les événements (créer, consulter, modifier et supprimer), Afin d'assurer la bonne organisation et la mise à jour du contenu du site de gestion des événements.

Ce cas de test vise à vérifier que les API sécurisées permettent correctement à l'administrateur d'effectuer toutes les opérations CRUD sur :

- les catégories
- les événements

via des requêtes HTTP (GET, POST, PUT, DELETE) testées avec Postman.

Titre	Gestion des catégories et des événements par l'administrateur
Description	Tester les opérations CRUD complètes via l'API REST pour les ressources Catégories et Événements, en vérifiant que l'administrateur peut effectuer toutes les opérations de gestion.
Technique de test	**Boîte noire** **Motivation du choix :** Les tests sont réalisés du point de vue fonctionnel, sans considération de l'implémentation interne. L'objectif est de vérifier que les API REST répondent correctement aux requêtes et retournent des résultats conformes aux exigences.
Niveau de test	**Test d'intégration** Les tests vérifient l'interaction entre : ➢ routes API, ➢ contrôleurs Laravel, ➢ règles de validation,

	➢ base de données MySQL.
Type de test	Test fonctionnel
Prérequis	➢ Serveur Laravel démarré ➢ Base de données avec données de test ➢ Administrateur authentifié ➢ Outil Postman configuré
Étapes	1. Authentifier l'administrateur et récupérer le token 2. Envoyer une requête HTTP vers l'API sécurisée 3. Fournir les données requises (JSON) 4. Vérifier la réponse retournée par le serveur
Méthode de test	Test manuel des endpoints API à l'aide de Postman, avec validation des réponses HTTP et des messages d'erreur retournés par le backend.

Tableau 10 : Fiche du cas de test « Gestion des catégories et des événements »

❖ Scénarios testés – Gestion des catégories

Scénario	Endpoint	Méthode	Résultat attendu	Résultat obtenu	Pass/Fail
Récupérer toutes les catégories	/api/categories	GET	Liste JSON des catégories avec code 200	Liste JSON des catégories avec code 200	PASS
Créer une nouvelle catégorie	/api/categories	POST	Catégorie créée avec id auto-incrémenté, code 201	Catégorie créée avec id auto-incrémenté, code 201	PASS
Modifier une catégorie existante	/api/categories/{id}	PUT	Catégorie mise à jour, code 200	Catégorie mise à jour, code 200	PASS
Supprimer une catégorie	/api/categories/{id}	DELETE	Message de confirmation, code 200	Message de confirmation, code 200	PASS

Tableau 11 : Scénarios testés – gestion des catégories

❖ Scénarios testés – Gestion des événements

Scénario	Endpoint	Méthode	Résultat attendu	Résultat obtenu	Pass/Fail
Récupérer tous les événements	/api/events	GET	Liste JSON des événements avec relations category, code 200	Liste JSON des événements avec relations category, code 200	PASS
Créer un nouvel événement	/api/events	POST	Événement créé avec id auto-incrémenté, code 201	Événement créé avec id auto-incrémenté, code 201	PASS
Modifier un	/api/events/{id}	PUT	Événement mis	Événement mis	PASS

événement existant			à jour, code 200	à jour, code 200	
Supprimer un événement	/api/events/{id}	DELETE	Message de confirmation, code 200	Message de confirmation, code 200	PASS

Tableau 12 : Scénarios testés – gestion des événements

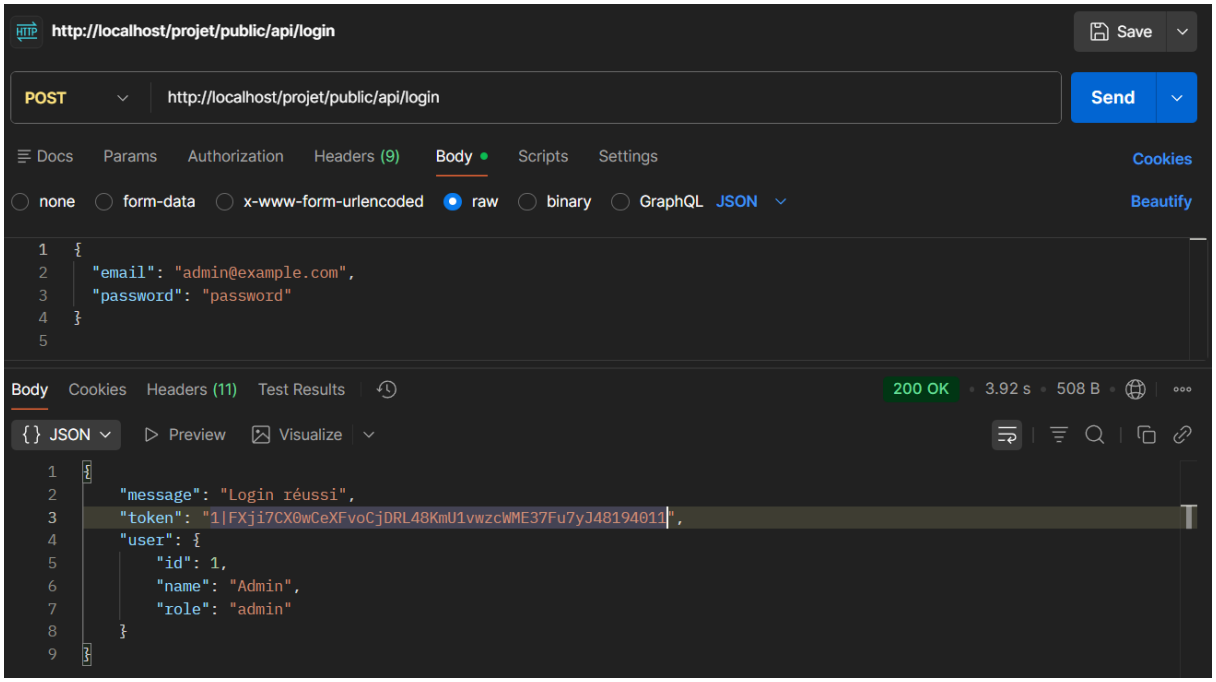
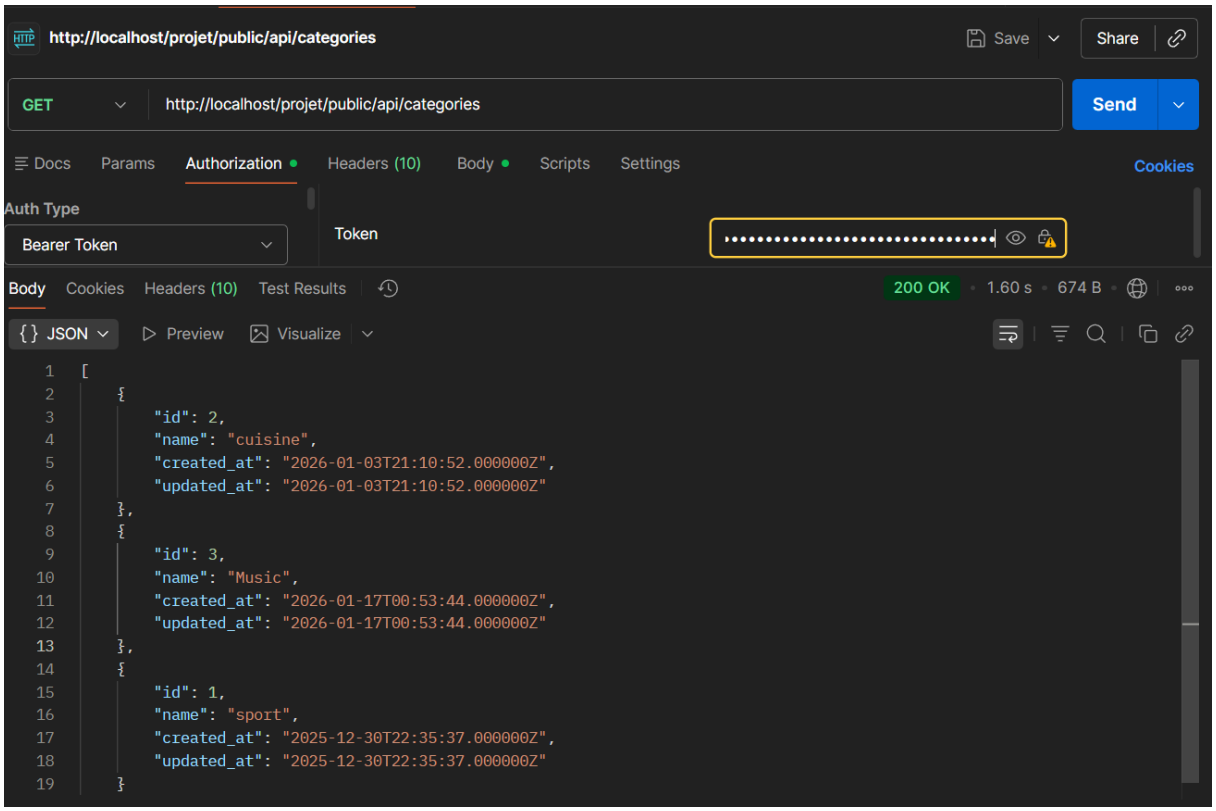
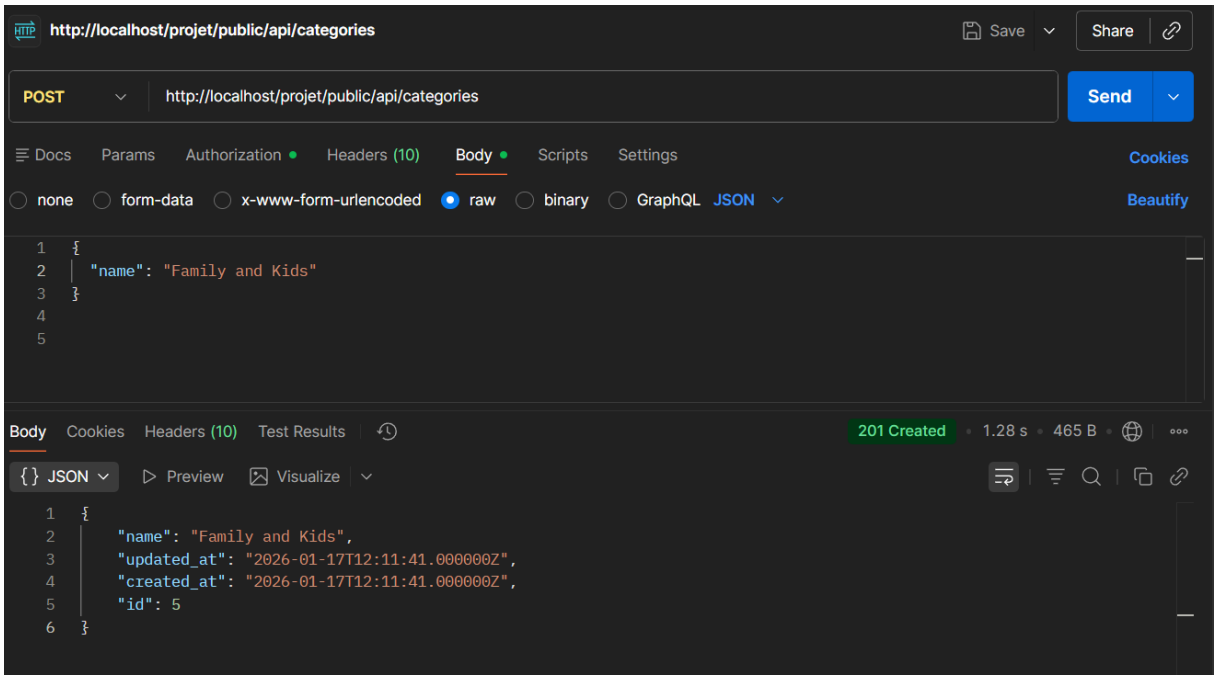


Figure 13 : Authentification de l'administrateur via /api/login et récupération du token


http://localhost/projet/public/api/categories
Save
Share
GET
http://localhost/projet/public/api/categories
Send
Docs
Params
Authorization
Headers (10)
Body
Scripts
Settings
Cookies
Auth Type
Bearer Token
Token
Body
Cookies
Headers (10)
Test Results
200 OK
1.60 s
674 B
JSON
Preview
Visualize
[
{
"id": 2,
"name": "cuisine",
"created_at": "2026-01-03T21:10:52.000000Z",
"updated_at": "2026-01-03T21:10:52.000000Z"
},
{
"id": 3,
"name": "Music",
"created_at": "2026-01-17T00:53:44.000000Z",
"updated_at": "2026-01-17T00:53:44.000000Z"
},
{
"id": 1,
"name": "sport",
"created_at": "2025-12-30T22:35:37.000000Z",
"updated_at": "2025-12-30T22:35:37.000000Z"
}


http://localhost/projet/public/api/categories
Save
Share
POST
http://localhost/projet/public/api/categories
Send
Docs
Params
Authorization
Headers (10)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"name": "Family and Kids"
}
Body
Cookies
Headers (10)
Test Results
201 Created
1.28 s
465 B
JSON
Preview
Visualize
{
"name": "Family and Kids",
"updated_at": "2026-01-17T12:11:41.000000Z",
"created_at": "2026-01-17T12:11:41.000000Z",
"id": 5
}

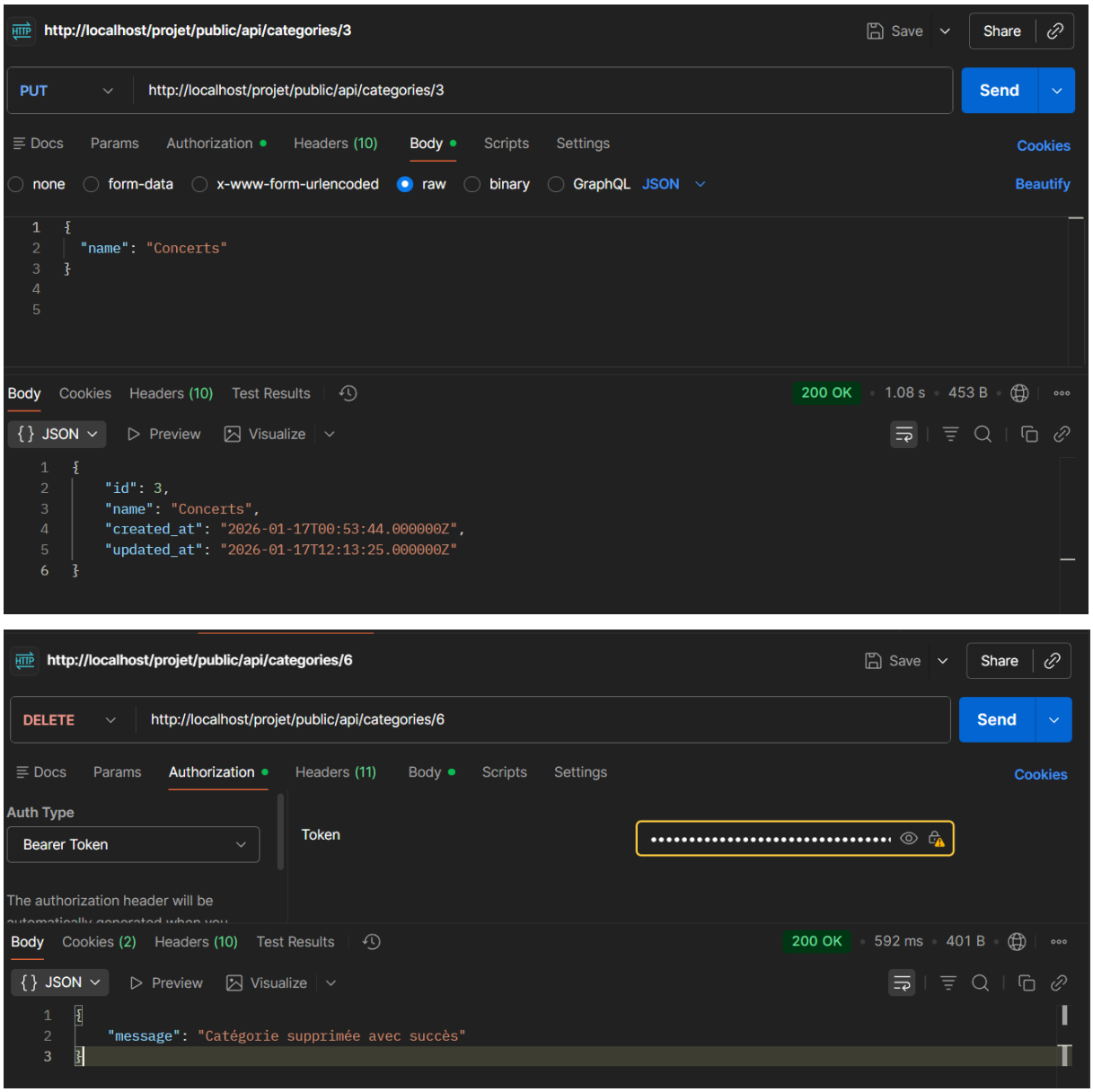
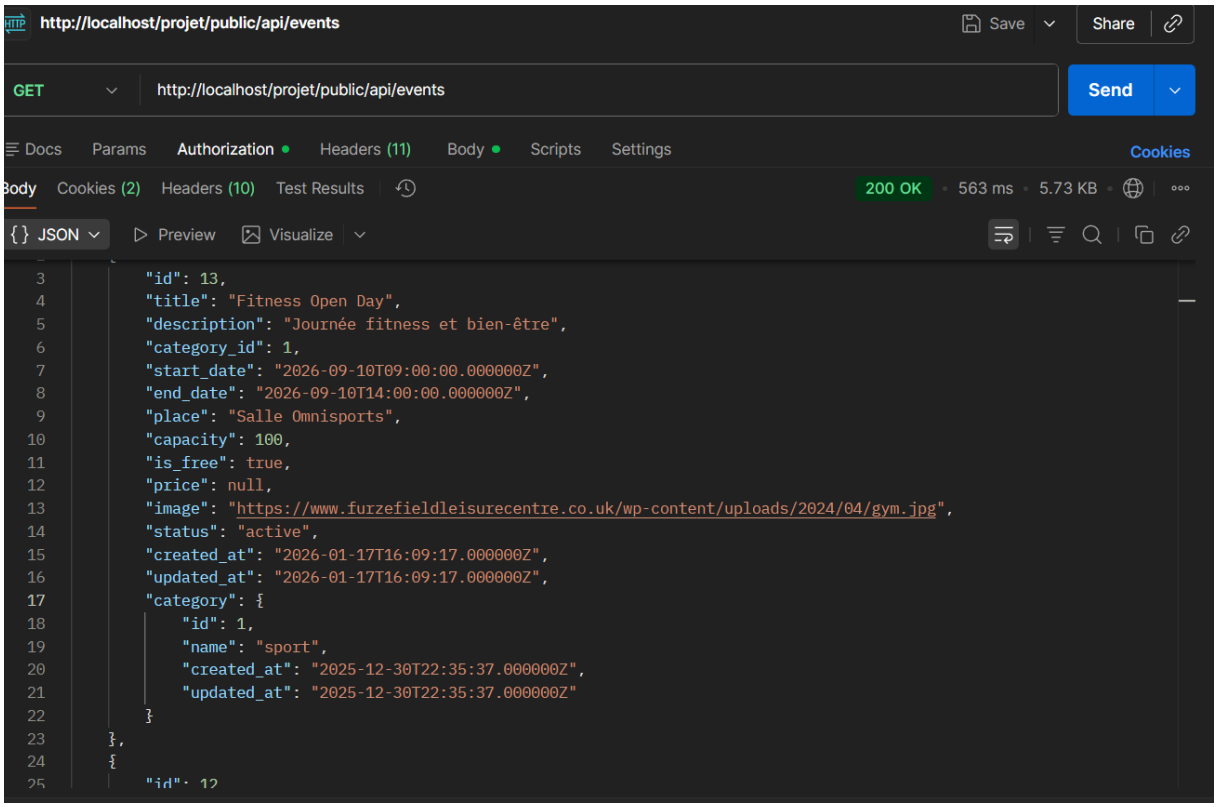
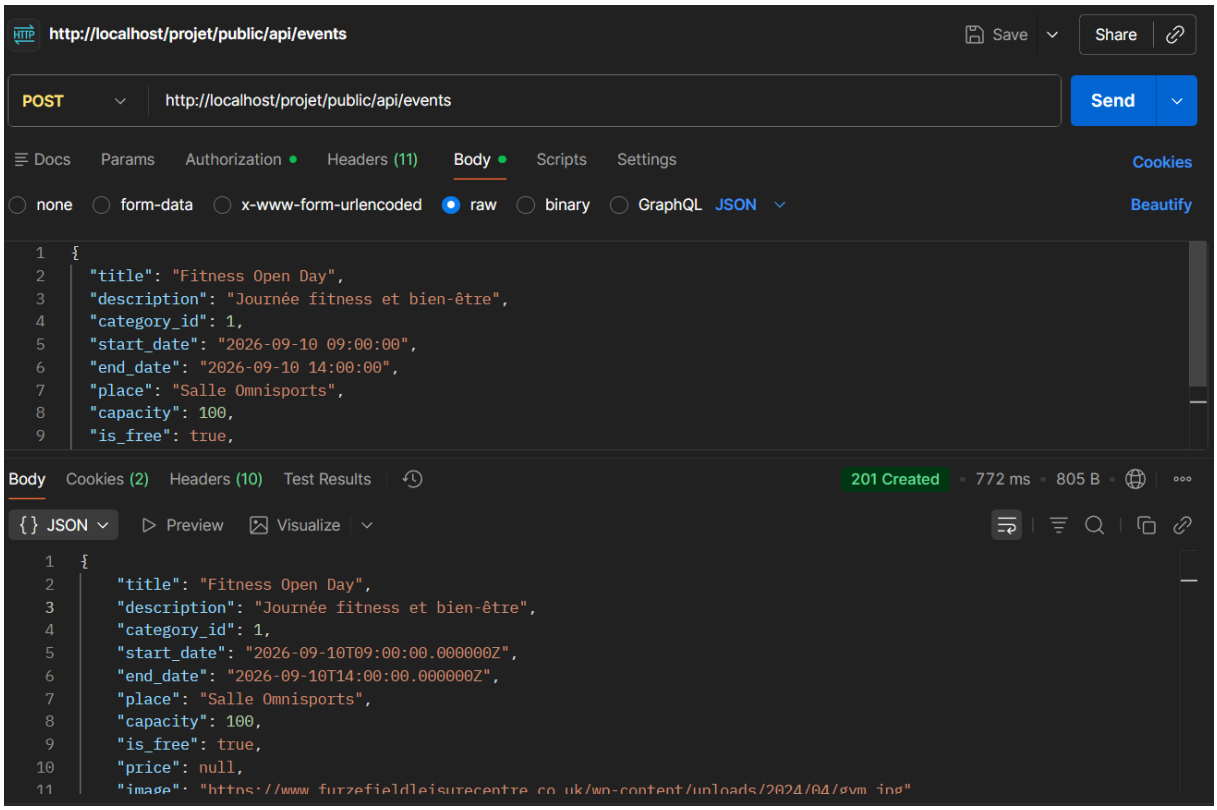


Figure 14 : Tests CRUD des catégories avec Postman (GET, POST, PUT, DELETE)


http://localhost/projet/public/api/events
Save
Share
GET
http://localhost/projet/public/api/events
Send
Docs
Params
Authorization
Headers (11)
Body
Scripts
Settings
Cookies
Body
Cookies (2)
Headers (10)
Test Results
200 OK
563 ms
5.73 KB
JSON
Preview
Visualize
"id": 13,
"title": "Fitness Open Day",
"description": "Journée fitness et bien-être",
"category_id": 1,
"start_date": "2026-09-10T09:00:00.000000Z",
"end_date": "2026-09-10T14:00:00.000000Z",
"place": "Salle Omnisports",
"capacity": 100,
"is_free": true,
"price": null,
"image": "https://www.furzefieldleisurecentre.co.uk/wp-content/uploads/2024/04/gym.jpg",
"status": "active",
"created_at": "2026-01-17T16:09:17.000000Z",
"updated_at": "2026-01-17T16:09:17.000000Z",
"category": {
"id": 1,
"name": "sport",
"created_at": "2025-12-30T22:35:37.000000Z",
"updated_at": "2025-12-30T22:35:37.000000Z"
}
},
{


http://localhost/projet/public/api/events
Save
Share
POST
http://localhost/projet/public/api/events
Send
Docs
Params
Authorization
Headers (11)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"title": "Fitness Open Day",
"description": "Journée fitness et bien-être",
"category_id": 1,
"start_date": "2026-09-10 09:00:00",
"end_date": "2026-09-10 14:00:00",
"place": "Salle Omnisports",
"capacity": 100,
"is_free": true,
Body
Cookies (2)
Headers (10)
Test Results
201 Created
772 ms
805 B
JSON
Preview
Visualize
{
"title": "Fitness Open Day",
"description": "Journée fitness et bien-être",
"category_id": 1,
"start_date": "2026-09-10T09:00:00.000000Z",
"end_date": "2026-09-10T14:00:00.000000Z",
"place": "Salle Omnisports",
"capacity": 100,
"is_free": true,
"price": null,

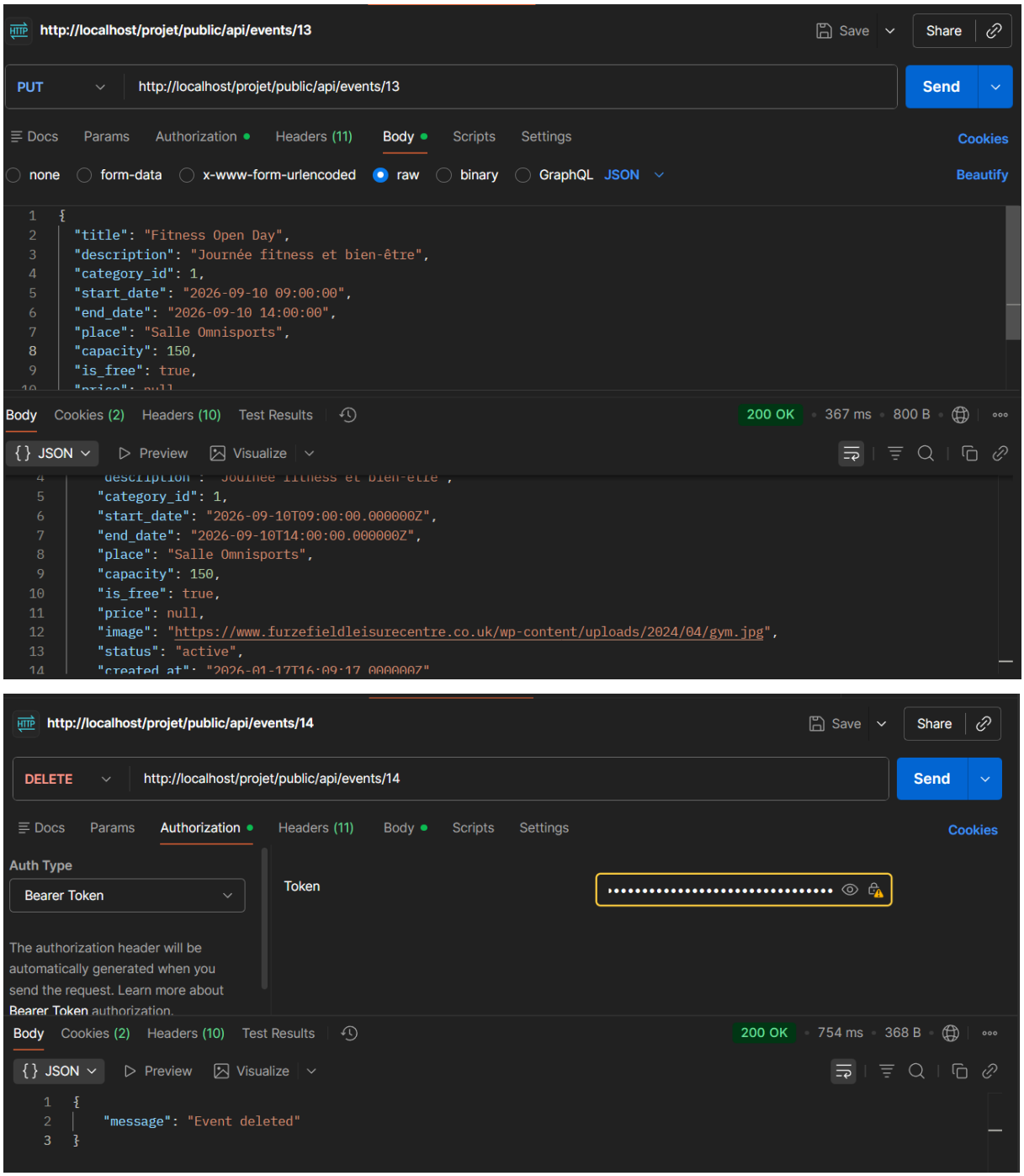


Figure 15 : Tests CRUD des événements avec Postman (GET, POST, PUT, DELETE)

Métrique	Valeur
Total des cas de test définis	4
Total tests exécutés	4
Tests réussis initialement	4 (100%)
Tests échoués initialement	0 (0%)
Tests réussis après correction	— (aucune correction nécessaire)
Taux d’exécution (exécutés / définis)	4/4 (100%)

Tableau 13 : Statistiques – gestion des catégories

Métrique	Valeur
Total des cas de test définis	4
Total tests exécutés	4
Tests réussis initialement	4 (100%)
Tests échoués initialement	0 (0%)
Tests réussis après correction	— (aucune correction nécessaire)
Taux d’exécution (exécutés / définis)	4/4 (100%)

Tableau 14 : Statistiques – gestion des événements

d. Documentation de Cas de Test : blocage des actions administrateur sans authentification:

❖ Description Générale

En tant qu'administrateur, Je veux pouvoir me connecter avec mes identifiants , Afin d'accéder au panneau d'administration.

Ce cas de test vise à vérifier que l’accès aux fonctionnalités de gestion des catégories et des événements (création, modification, suppression) est strictement réservé aux utilisateurs authentifiés ayant le rôle Administrateur.

Titre	Blocage des actions administrateur sans authentification.
Description	Ce test vérifie que toutes les opérations CRUD (Create, Read, Update, Delete) sur les ressources administrateur (catégories et événements) sont protégées par une authentification stricte.
Technique de test	**Boîte noire** **Motivation du choix :** Le test se base uniquement sur le comportement

	observable du système (réponse HTTP), sans connaissance de l'implémentation interne.
Niveau de test	**Test d'intégration (API)** il teste l'interaction entre le middleware d'authentification, les contrôleurs et l'API REST
Type de test	Test non fonctionnel (Sécurité)
Prérequis	➢ Serveur Laravel démarré ➢ Page de connexion disponible à /login ➢ L'admin n'est pas authentifié ➢ Aucun token d'authentification fourni dans la requête ➢ Header « Accept: application/json » ajouté dans Postman, pour que Laravel réponde en JSON
Étapes	1. Créer une requête HTTP (POST / PUT / DELETE) vers un endpoint administrateur 2. Ne pas fournir de token d'authentification dans le header 3. Envoyer la requête
Méthode de test	Test manuel des endpoints API à l'aide de Postman, avec validation des réponses HTTP et des messages d'erreur retournés par le backend.

Tableau 15 : Fiche du cas de test « Blocage des actions administrateur sans authentification »

❖ Scénarios testés

Scénario	Endpoint	Méthode	Résultat attendu	Résultat obtenu	Pass/Fail
Consultation des événements sans authentification	/api/events	GET	401 Unauthorized	401 Unauthorized	PASS
Création d'un événement sans authentification	/api/events	POST	401 Unauthorized	401 Unauthorized	PASS
Modification d'un événement sans authentification	/api/events/{id}	PUT	401 Unauthorized	401 Unauthorized	PASS
Suppression d'un événement sans authentification	/api/events/{id}	DELETE	401 Unauthorized	401 Unauthorized	PASS
Consultation des catégories sans authentification	/api/categories	GET	401 Unauthorized	401 Unauthorized	PASS
Création d'une catégorie sans authentification	/api/categories	POST	401 Unauthorized	401 Unauthorized	PASS
Modification d'une catégorie sans authentification	/api/ categories/{id}	PUT	401 Unauthorized	401 Unauthorized	PASS
Suppression d'une catégorie sans authentification	/api/ categories/{id}	DELETE	401 Unauthorized	401 Unauthorized	PASS

Tableau 16 : Scénarios testés – accès sans authentification

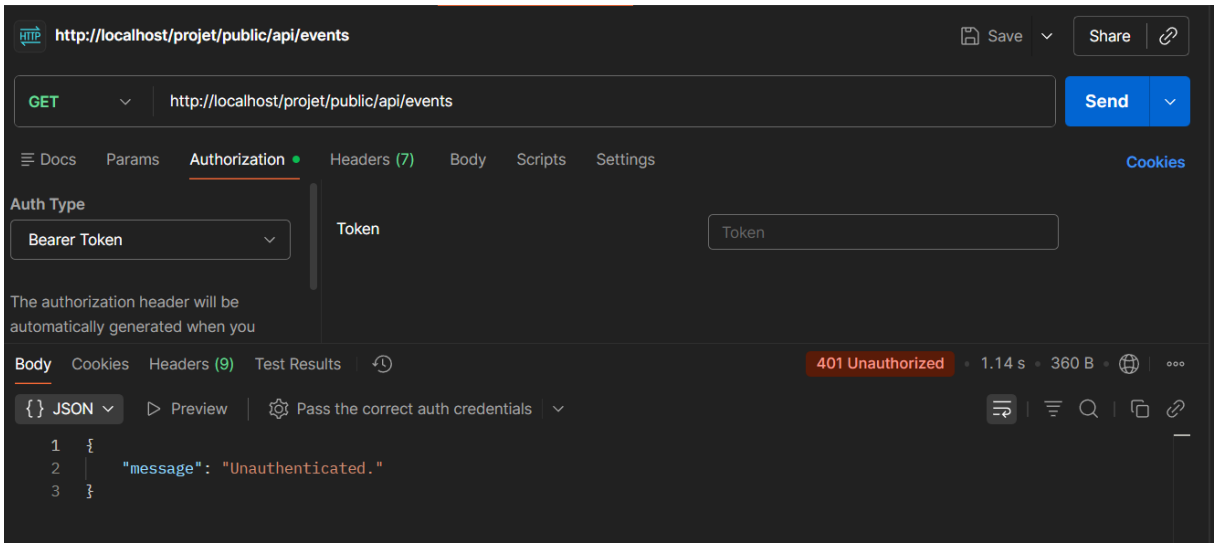


Figure 16 : Consultation des événements sans token : réponse 401 Unauthorized

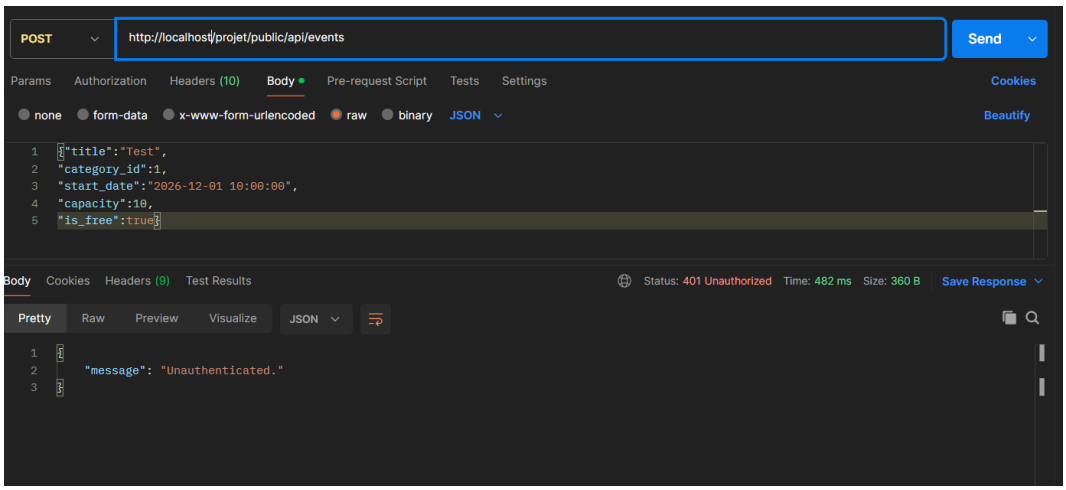


Figure 17 : Création d'un événement sans token : réponse 401 Unauthorized

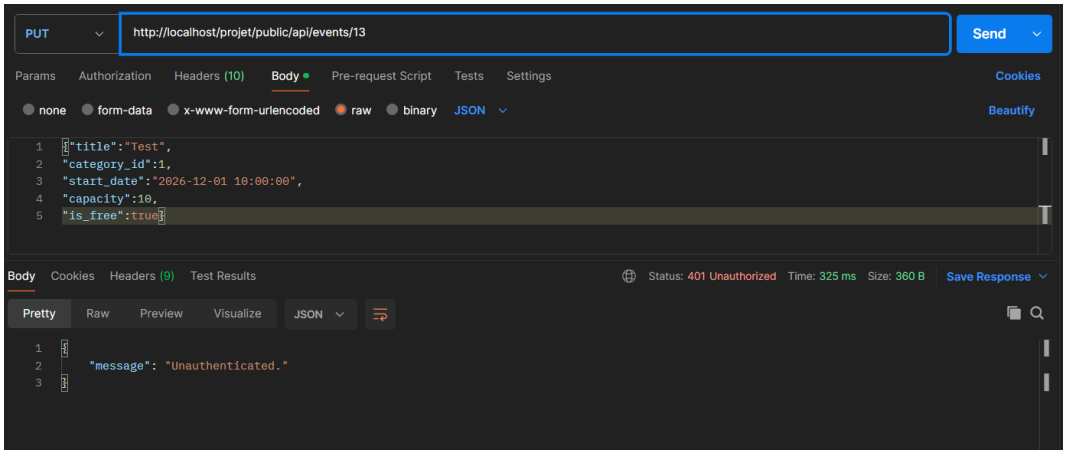


Figure 18 : Modification d'un événement sans token : réponse 401 Unauthorized

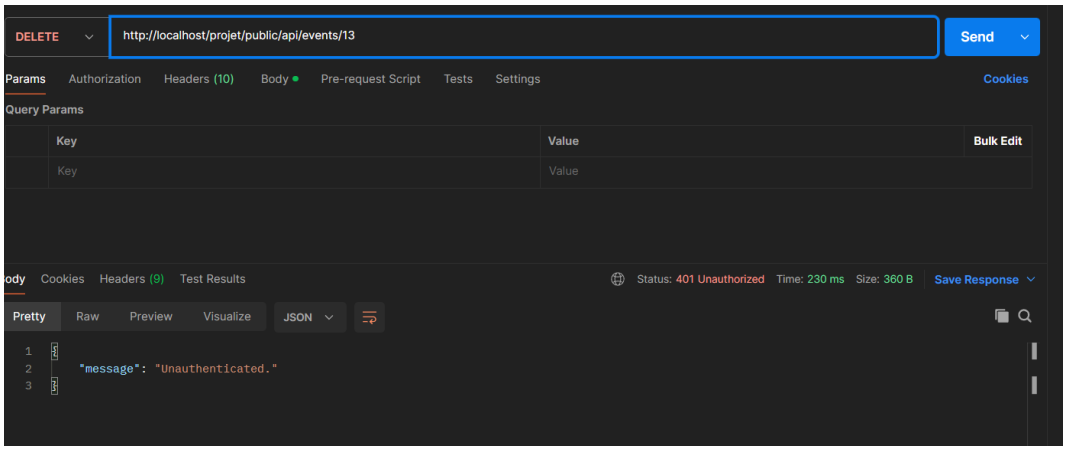


Figure 19 : Suppression d'un événement sans token : réponse 401 Unauthorized

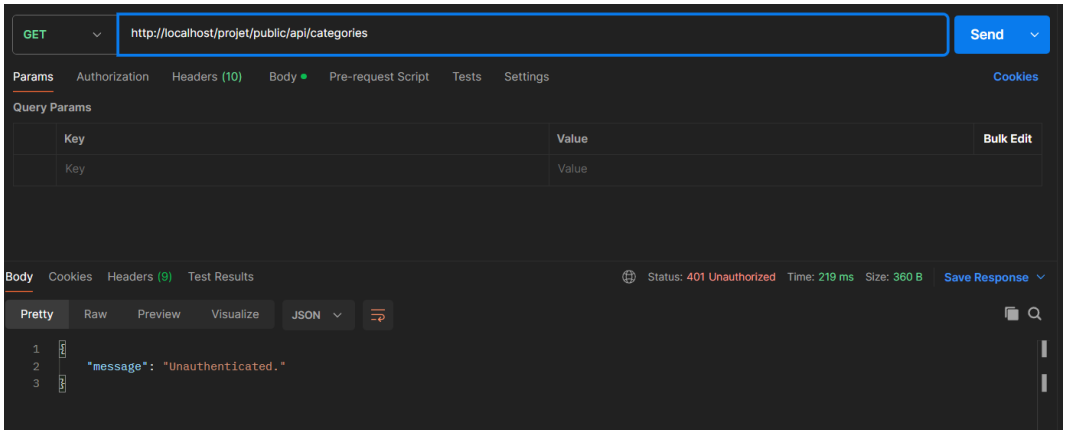


Figure 20 : Consultation des catégories sans token : réponse 401 Unauthorized

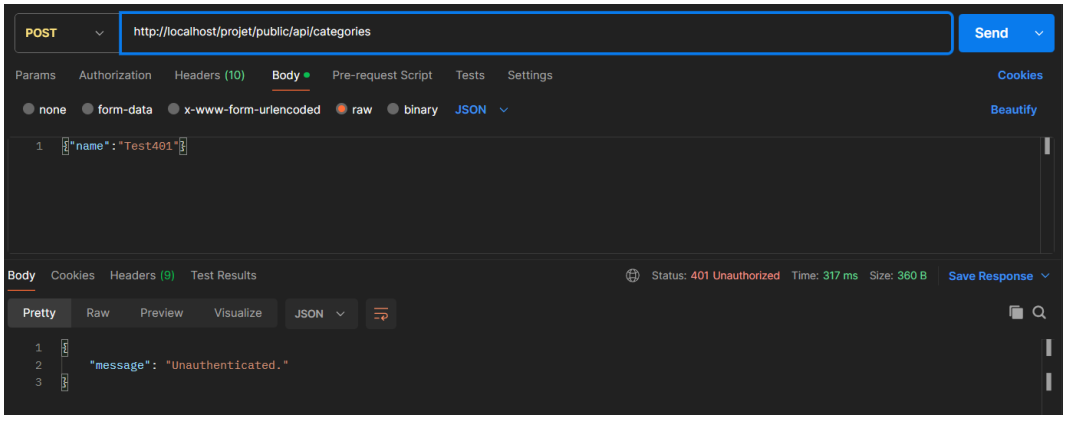


Figure 21 : Création d'une catégorie sans token : réponse 401 Unauthorized

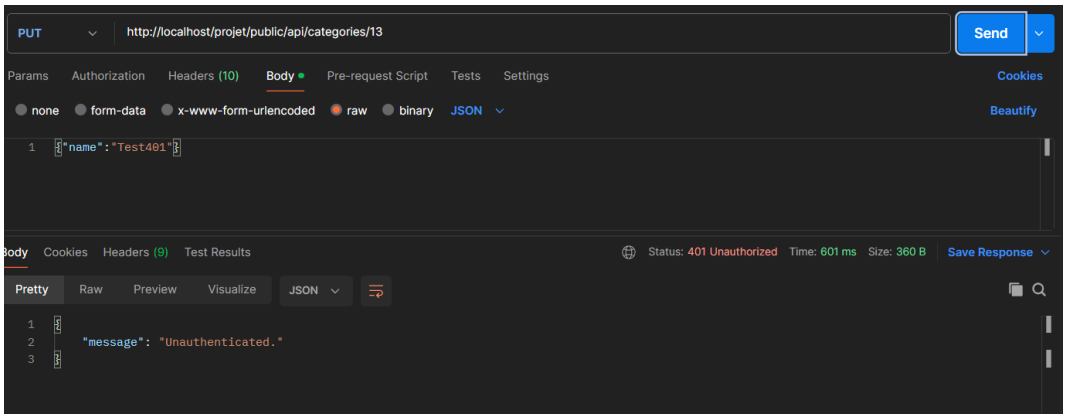


Figure 22 : Modification d'une catégorie sans token : réponse 401 Unauthorized

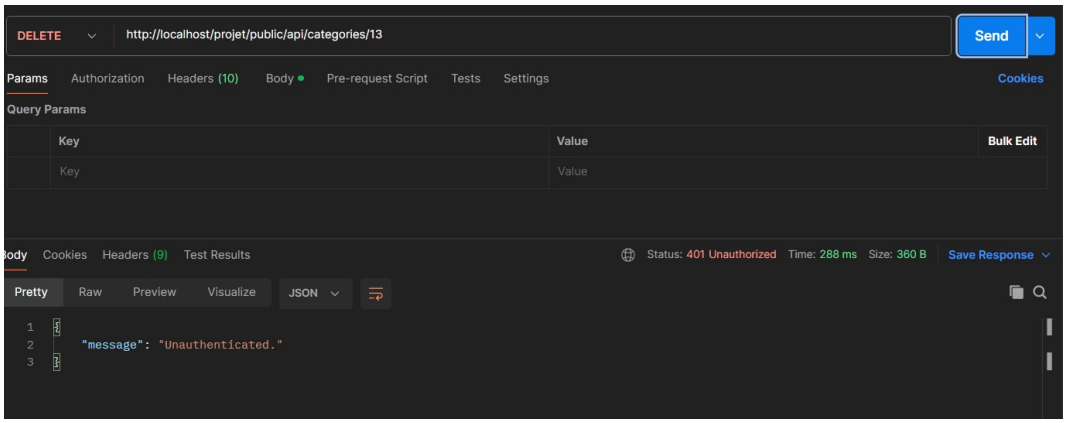


Figure 23 : Suppression d'une catégorie sans token : réponse 401 Unauthorized

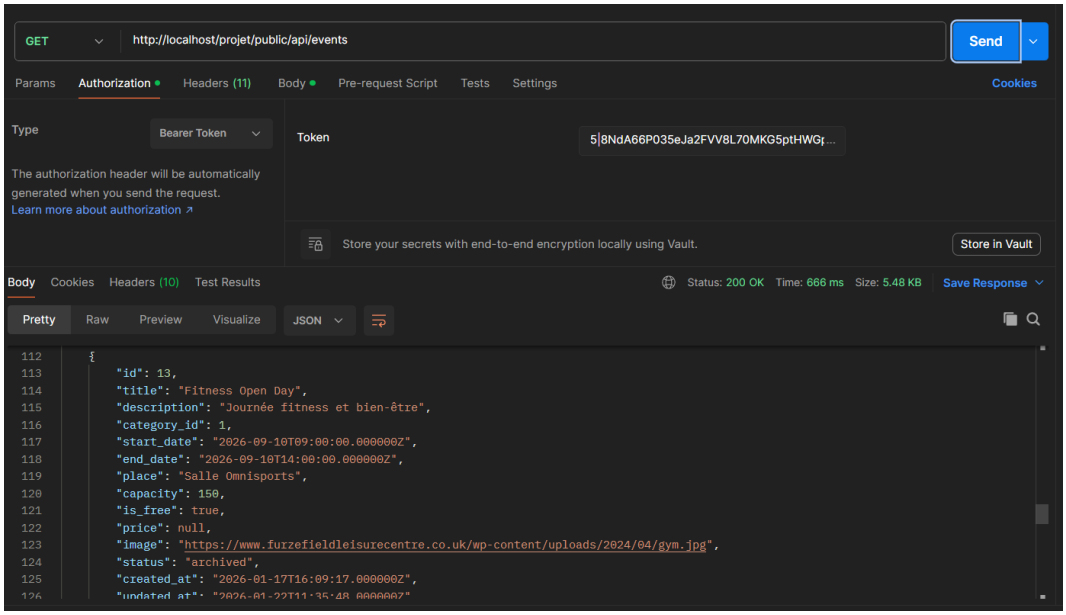


Figure 24 : Vérification avec le token administrateur : l'événement 13 existe toujours

Métrique	Valeur
Total des cas de test définis	8
Total tests exécutés	8
Tests réussis initialement	8 (100%)
Tests échoués initialement	0 (0%)
Tests réussis après correction	— (aucune correction nécessaire)
Taux d'exécution (exécutés / définis)	8/8 (100%)

Tableau 17 : Statistiques – accès sans authentification

e. Documentation de Cas de Test : Unicité du nom de catégorie:

❖ Description Générale

Titre	Unicité du nom de catégorie
Description	Ce test vérifie que le système empêche la création de deux catégories ayant le même nom, afin de garantir l'intégrité des données et éviter les doublons.
Technique de test	**Boîte noire** **Motivation du choix :** Le test est réalisé uniquement à partir des entrées utilisateur et des réponses du système, sans accès à l'implémentation interne.
Niveau de test	**Test d'intégration (API)** il vérifie l'interaction entre la validation backend, le contrôleur et la base de données
Type de test	Test non fonctionnel (Intégrité des données)
Prérequis	➢ Le serveur Laravel est démarré ➢ L'administrateur est authentifié (token valide fourni dans Postman) ➢ Une catégorie avec le même nom existe déjà dans la base
Étapes	1. Authentifier l'administrateur et récupérer le token 2. Créer une requête POST vers /api/categories 3. Fournir dans le body un nom de catégorie déjà existant 4. Envoyer la requête
Méthode de test	Test manuel des endpoints API à l'aide de Postman, avec validation des réponses HTTP et des messages d'erreur retournés par le backend.

Tableau 18 : Fiche du cas de test « Unicité du nom de catégorie »

❖ Scénarios testés

Scénario	Endpoint	Méthode	Résultat attendu	Résultat obtenu	Pass/Fail
Créer une catégorie avec un nom déjà existant	/api/categories	POST	422 Unprocessable Content	422 Unprocessable Content	PASS

Modifier une catégorie avec un nom déjà existant (ID différent)	/api/categories/{id}	PUT	422 Unprocessable Content	422 Unprocessable Content	PASS
Modifier une catégorie en conservant le même nom (même ID)	/api/categories/{id}	PUT	200 OK (la règle unique ignore la catégorie elle-même)	200 OK	PASS
Créer une catégorie avec variation de casse	/api/categories	POST	422 Unprocessable Content	422 Unprocessable Content	PASS

Tableau 19 : Scénarios testés – unicité du nom de catégorie

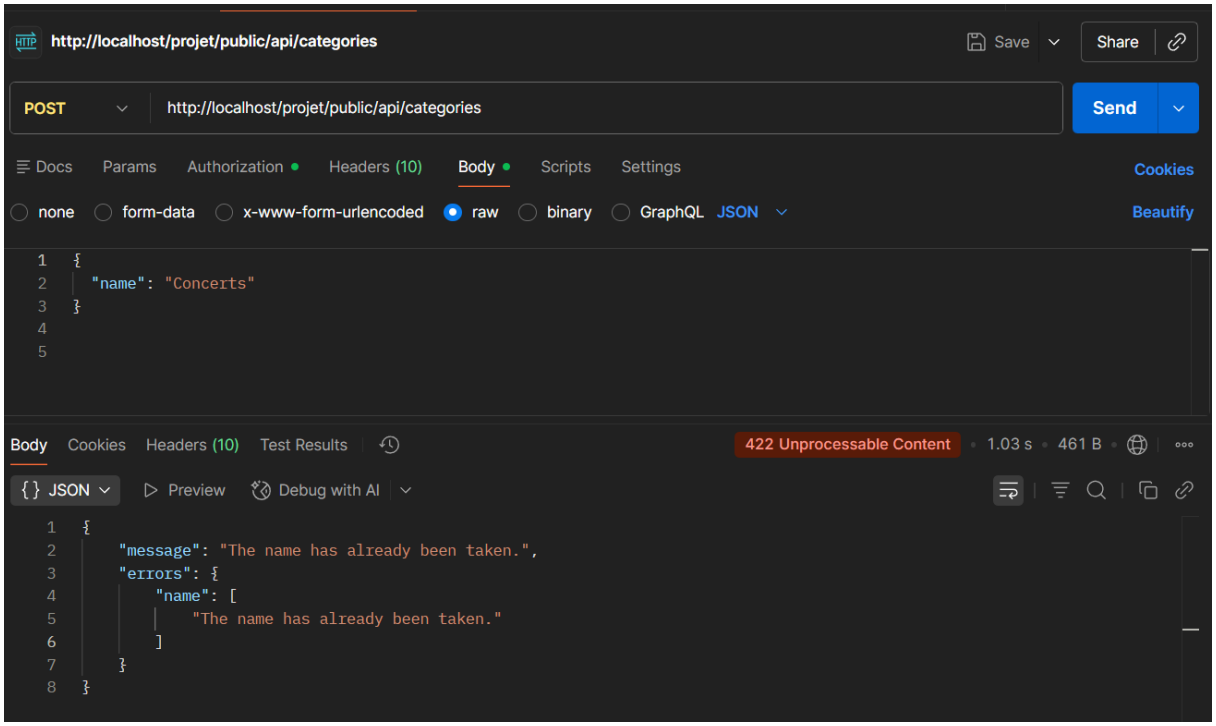


Figure 25 : Création d'une catégorie avec un nom déjà existant : réponse 422

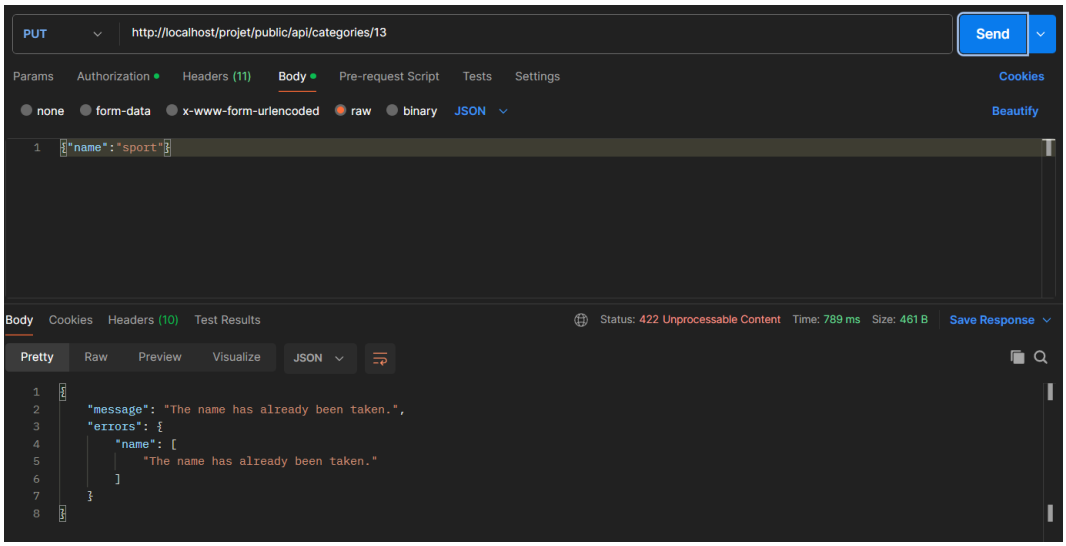


Figure 26 : Modification d'une catégorie avec le nom d'une autre catégorie : réponse 422

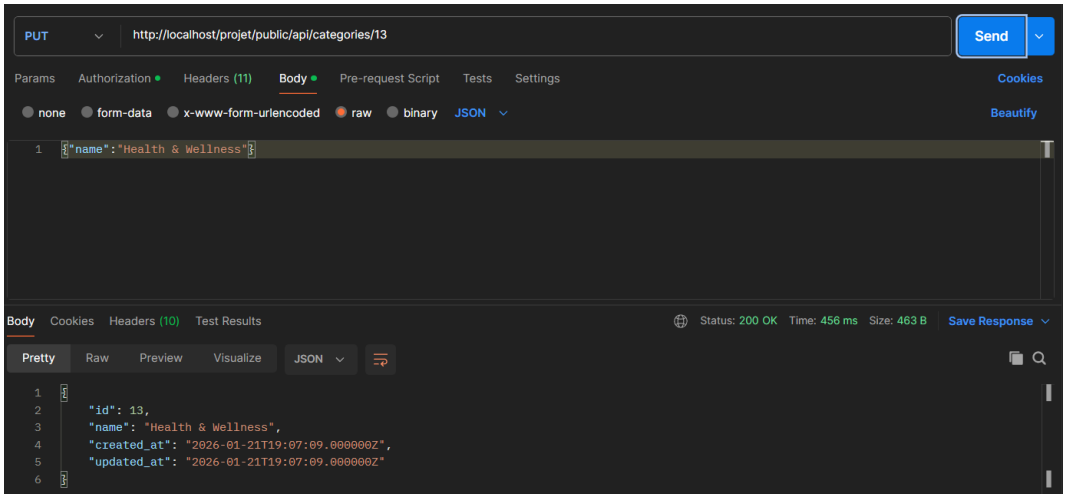


Figure 27 : Modification d'une catégorie en conservant son propre nom : réponse 200 OK

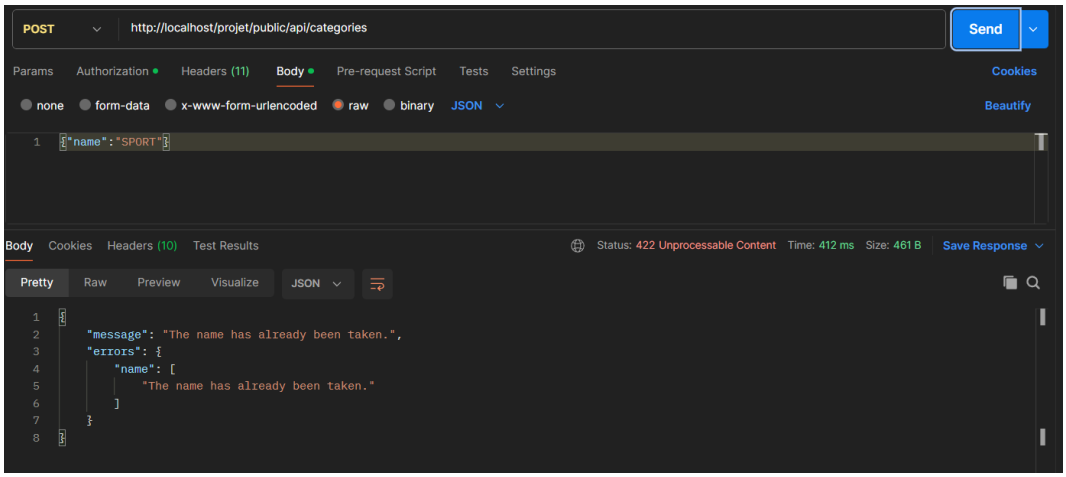


Figure 28 : Création d'une catégorie avec une variation de casse (« SPORT ») : réponse 422

Métrique	Valeur
Total des cas de test définis	4
Total tests exécutés	4
Tests réussis initialement	4 (100%)
Tests échoués initialement	0 (0%)
Tests réussis après correction	— (aucune correction nécessaire)
Taux d’exécution (exécutés / définis)	4/4 (100%)

Tableau 20 : Statistiques – unicité du nom de catégorie

f. Documentation de Cas de Test : Compréhensibilité des Messages d'Erreur

❖ Description Générale

En tant qu' utilisateur non technique, Je veux comprendre immédiatement pourquoi mon inscription a échoué, Afin de pouvoir corriger facilement mes erreurs et compléter mon inscription.

Titre	Compréhensibilité des Messages d'Erreur
Description	Évaluer si les messages d'erreur du formulaire d'inscription sont clairs, spécifiques, actionnables et compréhensibles par un utilisateur non technique.
Technique de test	**Boîte noire** **Motivation du choix :** Le test est réalisé du point de vue de l'utilisateur final, sans connaissance de l'implémentation interne. L'évaluation se base uniquement sur l'observation du comportement de l'interface et des messages affichés.
Niveau de test	**Test système** Le test couvre l'interface utilisateur complète (formulaire d'inscription) et son interaction avec la validation backend.
Type de test	Test non fonctionnel (Utilisabilité)
Prérequis	➢ Serveur Laravel démarré ➢ L'application est accessible et fonctionnelle ➢ L'utilisateur n'est pas authentifié ➢ Le formulaire d'inscription est disponible
Étapes	1. Accéder à la page d'inscription /register 2. Saisir des données invalides dans le formulaire selon le scénario testé 3. Cliquer sur le bouton "Sign Up" 4. Observer les messages d'erreur affichés par le système
Méthode de test	➢ Test manuel assisté par exécution du formulaire d'inscription ➢ Observation visuelle des messages d'erreur générés par le système

Tableau 21 : Fiche du cas de test « Compréhensibilité des messages d'erreur »

❖ Scénarios testés

Scénario	Résultat attendu	Résultat obtenu	Pass/Fail
Email déjà existant	"The email has already been taken"	"The email has already been taken"	PASS
Confirmation du mot de passe différente	"The password field confirmation does not match"	"The password field confirmation does not match"	PASS
Format email invalide (message du navigateur, validation HTML5)	"Please include an '@' in the email address. 'mail_invalide' is missing an '@'"	"Please include an '@' in the email address. 'mail_invalide' is missing an '@'"	PASS
Champs requis vides	"The name field is required. The email field is required. The	"The name field is required. The email field is	PASS

	password field is required”	required. The password field is required”	
Mot de passe ne respectant pas les critères	"The password is invalid. It must contain at least 6 characters, no more than 255 characters, include at least one uppercase letter, and at least one number."	“The password field format is invalid”	FAIL

Tableau 22 : Scénarios testés – messages d’erreur

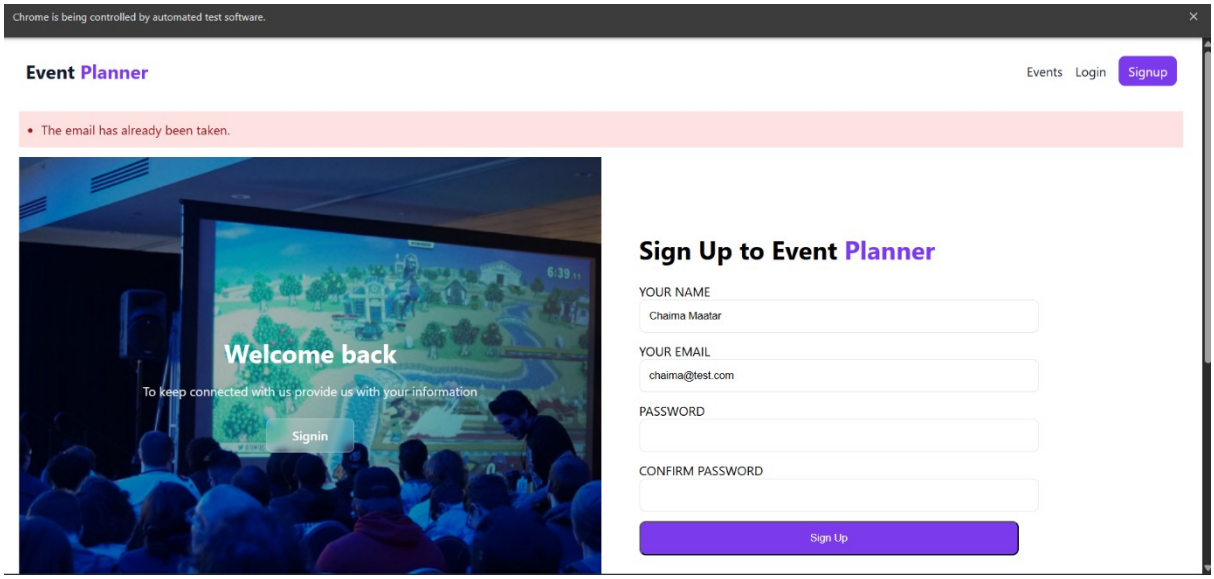


Figure 29 : Message affiché pour un email déjà existant

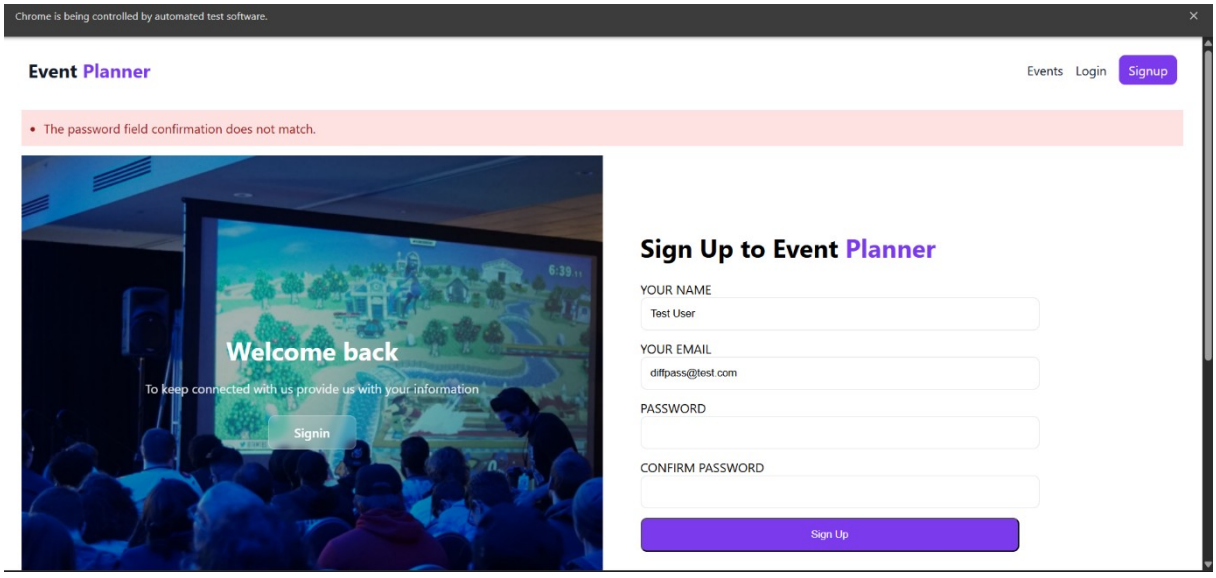


Figure 30 : Message affiché pour une confirmation du mot de passe différente

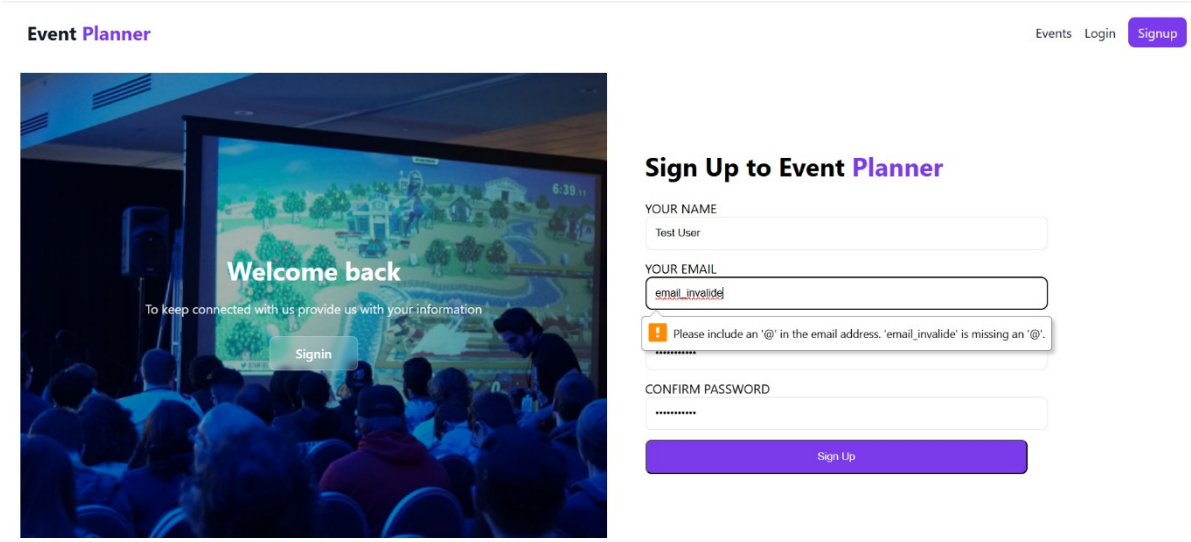


Figure 31 : Message du navigateur pour un format d'email invalide

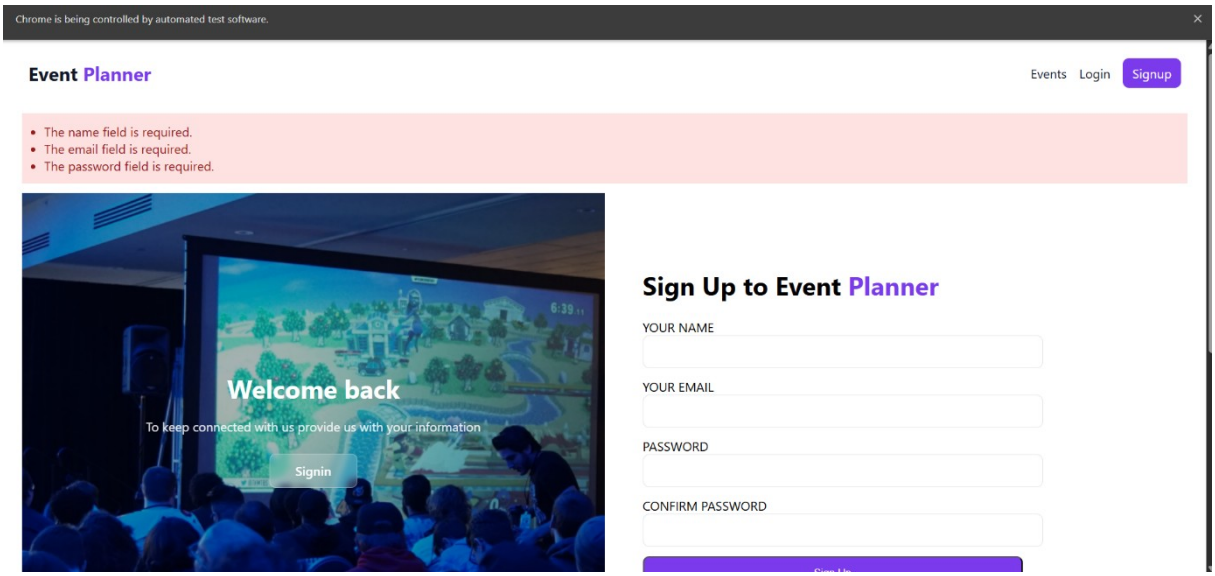


Figure 32 : Messages affichés pour des champs requis vides

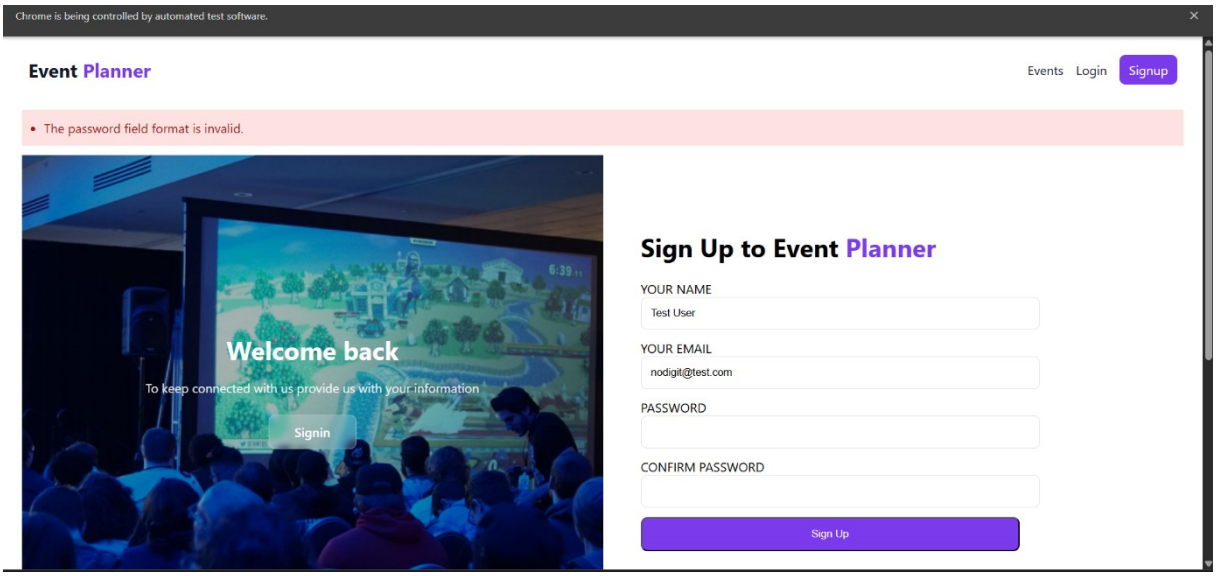


Figure 33 : Message affiché pour un mot de passe ne respectant pas les critères

Métrique	Valeur
Total des cas de test définis	5
Total tests exécutés	5
Tests réussis initialement	4 (80%)
Tests échoués initialement	1 (20%)
Tests réussis après correction	4/5 (80%) – message non modifié
Taux d'exécution (exécutés / définis)	5/5 (100%)

Tableau 23 : Statistiques – messages d'erreur

g. Documentation de Cas de Test : Protection CSRF des formulaires

❖ Description Générale

En tant qu'administrateur du système, Je veux m'assurer que tous les formulaires de l'application sont protégés contre les attaques CSRF (Cross-Site Request Forgery), Afin d'empêcher l'exécution d'actions non autorisées à l'insu de l'utilisateur authentifié.

Titre	Protection CSRF des formulaires
Description	Ce cas de test vise à vérifier, au niveau du code source, que Laravel applique correctement la protection CSRF via des tokens sur les formulaires et les requêtes sensibles.
Technique de test	**Boîte blanche** **Motivation du choix :** Le test est réalisé en analysant directement les vues Blade, Il ne repose pas sur le comportement visible côté utilisateur, mais sur la présence effective des mécanismes de sécurité dans le code.
Niveau de test	**Test statique (inspection du code source)** Aucune exécution de scénario utilisateur n'est nécessaire.
Type de test	Test non fonctionnel (Sécurité)
Prérequis	➢ Le code source de l'application Laravel est accessible ➢ Les fichiers de vues Blade (.blade.php) sont disponibles
Étapes	1. Identifier toutes les vues Blade contenant des formulaires HTML sensibles (authentification, création, modification, suppression). 2. Ouvrir chaque fichier Blade correspondant. 3. Vérifier la présence de la directive @csrf dans chaque formulaire
Méthode de test	Inspection statique du code source Laravel

Tableau 24 : Fiche du cas de test « Protection CSRF des formulaires »

❖ Scénarios testés

Formulaire	Fichier	@csrf présent	Pass/Fail
Inscription	auth/register.blade.php	Oui	PASS
Connexion	auth/login.blade.php	Oui	PASS
Création événement	admin/events/create.blade.php	Oui	PASS
Édition événement	admin/events/edit.blade.php	Oui	PASS
Création catégorie	admin/categories/create.blade.php	Oui	PASS
Édition catégorie	admin/categories/edit.blade.php	Oui	PASS
Liste catégories	admin/categories/index.blade.php	Oui	PASS
Liste événements (admin)	admin/events/index.blade.php	Oui	PASS
Liste événements (public)	events/index.blade.php	Non (formulaire GET)	PASS
Détail événement	events/show.blade.php	Oui	PASS
Layout administrateur	components/layouts/admin.blade.php	Oui	PASS
Layout application	components/layouts/app.blade.php	Oui	PASS

Tableau 25 : Présence de la directive @csrf par formulaire

```
<h1>Create category</h1>
<form method="POST" action="{{ route('admin.categories.store') }}" style="max-width:480px;">
    @csrf
```

Figure 34 : @csrf dans admin/categories/create.blade.php

```
<form method="POST" action="{{ route('admin.categories.destroy', $category) }}" style="margin-top:16px;">
    @csrf
    @method('DELETE')
```

Figure 35 : @csrf dans admin/categories/edit.blade.php

```
<form method="POST" action="{{ route('admin.categories.destroy', $category) }}" style="margin:0;">
    @csrf
    @method('DELETE')
```

Figure 36 : @csrf dans admin/categories/index.blade.php

```
<h1 style="color:☐#111827;font-size:32px;font-weight:bold;text-align:center;margin:0 0 24px;">Create Event</h1>
<form method="POST" action="{{ route('admin.events.store') }}" style="max-width:900px;margin:0 auto;">
    @csrf
```

Figure 37 : @csrf dans admin/events/create.blade.php

```
<h1 style="color:☐#111827;font-size:32px;font-weight:bold;text-align:center;margin:0 0 24px;">Edit Event</h1>
<form method="POST" action="{{ route('admin.events.update', $event) }}" style="max-width:900px;margin:0 auto;">
    @csrf
    @method('PUT')
```

Figure 38 : @csrf dans admin/events/edit.blade.php

```blade
<form method="POST" action="{{ route('admin.events.archive', $event) }}" style="margin:0;">
    @csrf
    @method('PUT')
```

Figure 39 : @csrf dans admin/events/index.blade.php

```blade
<form method="POST" action="{{ url('/login') }}" style="max-width:480px;">
    @csrf
    <div style="margin-bottom:12px;">
```

Figure 40 : @csrf dans auth/login.blade.php

```blade
<form method="POST" action="{{ url('/register') }}" style="max-width:480px;">
    @csrf
    <div style="margin-bottom:12px;">
```

Figure 41 : @csrf dans auth/register.blade.php

```blade
<form method="POST" action="{{ route('logout') }}" style="margin:0;border-top:1px solid #f3f4f6;">
  @csrf
  <button id="logout-btn-admin" type="submit" style="display:block;width:100%;text-align:left;padding:10px 12px;border:none;background:white;color
</form>
```

Figure 42 : @csrf dans components/layouts/admin.blade.php

```blade
<form method="POST" action="{{ route('logout') }}" style="margin:0;">
    @csrf
    <button id="logout-btn-app" type="submit" style="display:block;width:100%;text-align
</form>
```

Figure 43 : @csrf dans components/layouts/app.blade.php

```blade
<form method="POST" action="{{ route('events.register', $event) }}" id="register-form">
    @csrf
</form>
```

Figure 44 : @csrf dans events/show.blade.php

```blade
<form method="GET" action="{{ route('home') }}" style="display:flex;gap:12px;">
    <div style="position:relative;">
        <input type="text" name="q" value="{{ request('q') }}" placeholder="Search"
        <button type="submit" style="position:absolute;right:8px;top:50%;transform:
```

Figure 45 : formulaire de recherche en GET, sans @csrf (non nécessaire), dans events/index.blade.php

Métrique	Valeur
Total des formulaires identifiés	12
Formulaires nécessitant un token CSRF	11
Formulaires avec @csrf présent	11
Formulaires sans @csrf justifié (GET public)	1
Tests réussis	12
Tests échoués	0
Taux d’exécution (exécutés / définis)	12/12 (100%)

Tableau 26 : Statistiques – protection CSRF

III. RAPPORT FINAL :

a. Fonctionnalités Administrateur

Exigence	Testé ?
Authentification Admin	oui
Visualiser les événements	oui
Création d'événement	oui
Modification d'événement	oui
Suppression d'événement	oui
Visualiser les catégories	oui
Création catégorie	oui
Modification catégorie	oui
Suppression catégories	oui
Visualisation inscriptions	non
Archive événement	non
Couverture	9/11

Tableau 27 : Couverture des fonctionnalités administrateur

b. Fonctionnalités Utilisateur

Exigence	Testé ?
Inscription site	oui
Connexion compte	oui
Consultation événements	non
Recherche événements	non
Filtrage événements	non
Détail événement	non
Inscription événement	non
Désinscription événement	non
Mes inscriptions	non
Visualisation inscriptions	non
Disponibilité places	non
Couverture	2/11

Tableau 28 : Couverture des fonctionnalités utilisateur

c. Tests Non-Fonctionnels

❖ Sécurité

Exigence	Testé ?
Hash bcrypt	non
Protection CSRF	oui
Contrôle accès admin	oui
Couverture	2/3

Tableau 29 : Couverture des exigences de sécurité

❖ Intégrité Données

Exigence	Testé ?
Unicité noms catégories	oui
Cascade delete catégorie	non
Dates start < end	non
Données manquantes	oui
Couverture	2/4

Tableau 30 : Couverture des exigences d'intégrité des données

❖ Utilisabilité

Exigence	Testé ?
Messages erreur compréhensibles	oui

Interface responsive	non
Couverture	1/2

Tableau 31 : Couverture des exigences d'utilisabilité